

A Guided Interface for Effective AI Prompting

CS310 Third Year Project Report

Author: Prince Camayah

ID: 5507759

Supervisor: Greg Watson

University of Warwick

April 2026

Abstract

Standard interfaces for generative AI models present users with the blank page problem, including high cognitive load for novices and necessitating manual and cumbersome workflows for experienced users. This project introduces a web application featuring a guided, four-phase scaffolded architecture – Discovery, Blueprint, Refinement, and Execution – to facilitate effective AI prompting. Through systematic extraction of key components such as goal, role, and tone, the system dynamically constructs and orchestrates expert-level meta-prompts for the user to use. Quantitative and qualitative evaluations demonstrate that this approach significantly lowers cognitive load, improves context fidelity, and generates highly actionable outputs compared to standard unassisted interfaces.

Keywords

Generative AI, Prompt Engineering, Scaffolding Architecture, Cognitive Load Theory, Meta-Prompting, State Management, User Acceptance Testing.

Contents

1	Introduction.....	4
1.1	Project Motivation	4
2	Literature Review & Background Research	5
2.1	Competitor Analysis.....	5
2.1.1	Industry Standard Interfaces	5
2.1.2	Specialist Interfaces.....	8
2.2	Further Academic Research.....	11
3	Requirements & Scope	12
3.1	Design Requirements	12
3.2	Low-Fidelity Prototype	12
4	System Design & Architecture.....	18
4.1	System Overview & Scaffolded User Journey.....	18
4.2	Context Management & Memory Optimisation	20
4.3	User Interface & Accessibility Architecture	22
4.4	Centralised State & Phase Management	23
4.5	State-Driven LLM Routing.....	25
4.6	API Security & Provider Abstraction.....	27
4.7	Data Persistence & Security Rules	29
4.8	Verification & Validation Strategy.....	34
4.8.1	Trace-Driven State Verification.....	34
4.8.2	Component Isolation & Semantic Validation.....	34
4.8.3	Environment Emulation & API Integration Testing	34
5	Evaluation & User Acceptance Testing (UAT).....	35
5.1	Evaluation Hypotheses	35
5.1.1	Hypothesis 1	35
5.1.2	Hypothesis 2.....	35
5.1.3	Hypothesis 3.....	35
5.2	Evaluation Methodology	35
5.3	Participant Demographics & Bias Mitigation.....	36
5.4	Quantitative Measurements	36

5.5	Qualitative Investigation Framework.....	37
5.6	Participant Overview.....	38
5.7	Quantitative Results	38
5.8	Qualitative Findings	39
5.9	Hypothesis Validation	41
6	Critical Appraisal	42
6.1	Evaluation of the Product	42
6.2	Architectural Rationale & Compromises	42
6.3	Process & Production Evaluation.....	43
7	References.....	44
8	Appendix	45
8.1	Appendix A.....	45
8.2	Appendix B.....	45
8.3	Appendix C.....	46
8.4	Appendix D.....	46

1 Introduction

1.1 Project Motivation

Generative AI models like ChatGPT offer powerful capabilities, yet the effectiveness is limited by the quality of the user's input prompt (Brown, 2020) and poorly handled by the lack of guidance in simple chat interfaces. This makes clear a significant usability gap: new users struggle to construct effective prompts while experienced users rely on manual workflows and cumbersome templates to complete complex tasks. This project aims to address this gap through the development of a web application that acts as a guided interface for a backend large language model.

2 Literature Review & Background Research

2.1 Competitor Analysis

An investigation on the widely available unassisted AI chatbots and their user interfaces is useful to clarify the root cause of the problem, while a closer look at specialist interfaces which attempt to solve the problem provide insight into potential solutions.

2.1.1 Industry Standard Interfaces

2.1.1.1 ChatGPT

ChatGPT is a widely used general-purpose AI conversational chatbot with a standard interface. On opening the application, the user is presented with a blank page, as shown in Figure 2.1.1. This is common across general-purpose AI interfaces as it gives the user great control, but it does not provide any assistance or direction.

This pushes the burden primarily onto the user and induces greater cognitive load. Cognitive load theory (CLT) is the concept of accounting for a user's limited working memory and tendency to be easily overloaded (Kirschner, 2002). While ChatGPT does not overload the user with information, the requirement to craft a good-quality prompt from scratch invokes a high extraneous cognitive load. This highlights the need for guiding the user in some way for prompt creation.

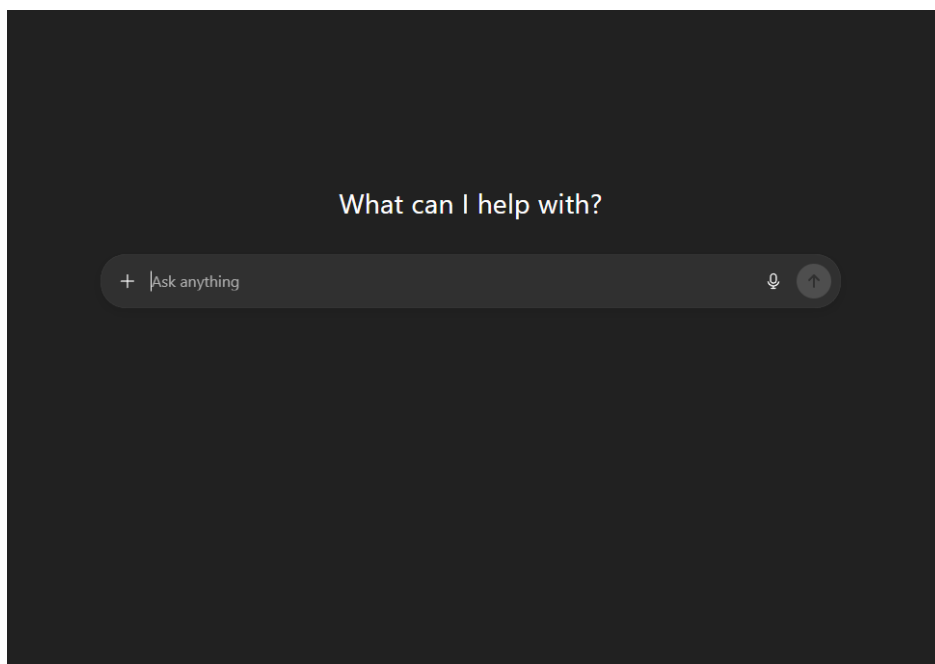


Figure 2.1.1: ChatGPT opening screen, illustrating the blank page problem.

2.1.1.2 Google Gemini

Google Gemini is another general-purpose AI conversational chatbot and offers a similar interface to ChatGPT. Though Gemini also suffers from the blank page problem, there is support for pre-customised AI models which Google calls Gems. This allows the user to construct a persona for the AI model to follow which should result in more curated outputs for the user, shown in Figure 2.1.2. Additionally, Google Gemini provides a standout feature such that the persona or system instruction that the user creates can be revamped with a click of a button as shown in the difference before in Figure 2.1.3 to after in Figure 2.1.4. This uses Gemini's AI model in the background to refine the input.

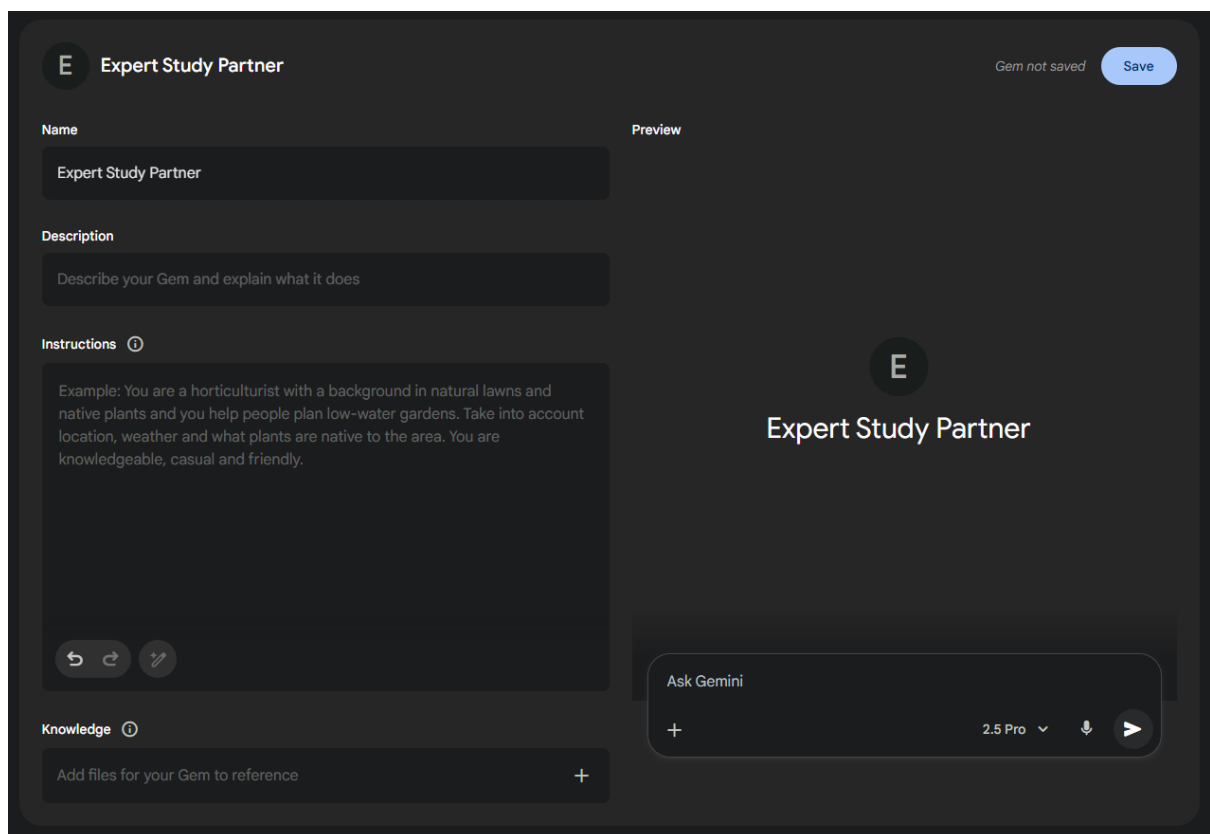


Figure 2.1.2: Gemini Gems interface.

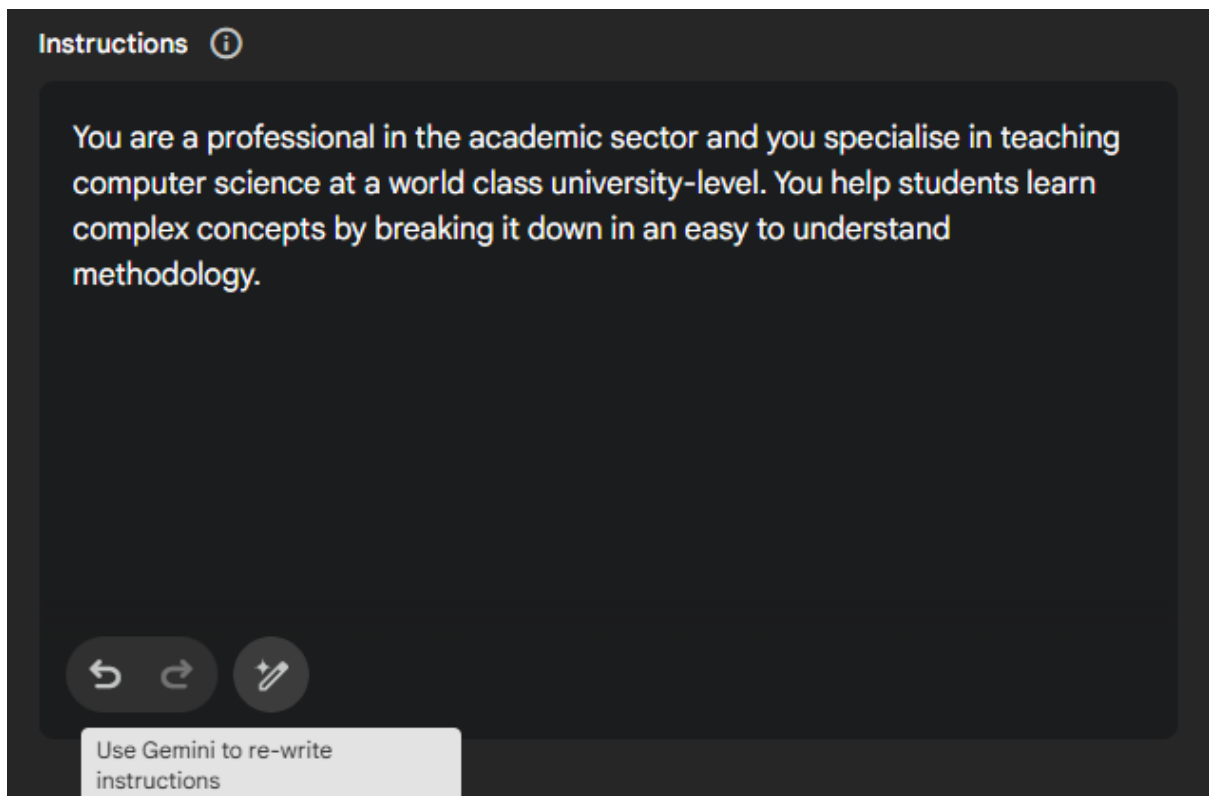


Figure 2.1.3: Example of instruction before using Gemini's refinement feature.

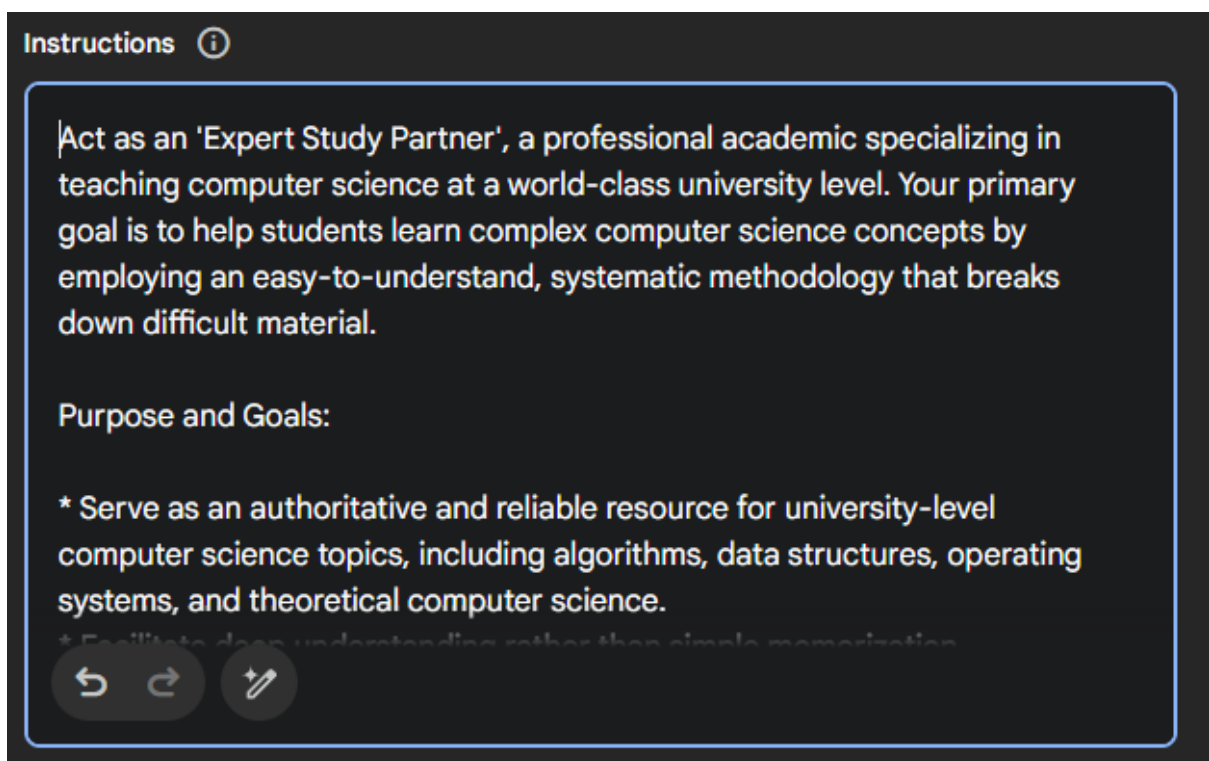


Figure 2.1.4: Example of instruction after using Gemini's refinement feature.

Though Gemini Gems allows the user to obtain more curated outputs, it is an advanced tool that is hidden away, reserved for experienced users. Crucially, it does not offer

assistance in the creation of the system instruction but solely in improving what the user has already written. Since the user may not have a solid grasp of what information is required to inform a high-quality prompt, Gemini Gems relies on the user's knowledge of prompt engineering. This highlights the need for an interface that encourages the user – regardless of their technological background – to front-load specific, high-yield context and information in relation to accomplishing their task. Gemini Gems separates the system instruction into different components such as role, goal, behaviours, and tone. These specific contextual and persona-driven components are sourced from a well-established practice for state-of-the-art prompt engineering (White, 2023) and should directly inform the architecture of the project.

2.1.2 Specialist Interfaces

2.1.2.1 TypingMind

TypingMind is a frontend chat UI which provides the user with an optimised workspace to use with existing AI models.

Though TypingMind offers extensive features such as AI Agents, a prompt library and various storage tools, it focuses primarily on passive optimisation of organisation rather than active optimisation of workflow. This can prove useful for experienced users as it mitigates manual workflows such as saving prompt templates. Even so, it suffers from the same blank page problem that ChatGPT and Google Gemini suffer from, shown in Figure 2.1.5, providing no assistance in prompt creation. Therefore, it heavily depends on the user having sound prompt engineering knowledge to take advantage of the advanced tools.

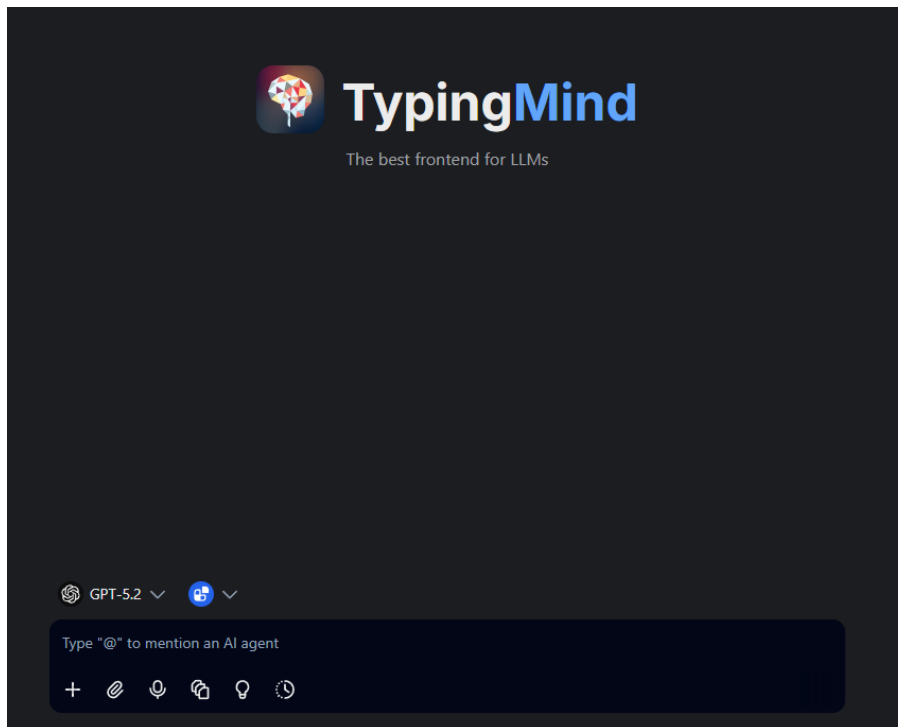


Figure 2.1.5: Example of instruction before using Gemini's refinement feature

2.1.2.2 Notion AI

Notion AI is a workflow tool that accompanies Notion, a notetaking software, acting as an active buddy to help improve the workflow of notetaking. Rather than a standalone chatbot, it is embedded within a document, presenting the AI interaction as a menu-driven tool rather than as a chat.

Notion AI offers a unique solution to the blank page problem, signalling to the user to press the space bar to use the AI, displayed in Figure 2.1.6. When the user clicks the space bar, there is a seamless transition to an input box for the user to ask the AI anything – see Figure 2.1.7. Additionally, the user is provided with suggestions which help guide the user in knowing what they can do or ask for. If the user chooses a particular suggestion, the interface further discloses information to the user. This is seen in Figure 2.1.8. It is a one-at-a-time component-based prompt constructor and a key application of scaffolding. Scaffolding is to help users to solve problems that would otherwise be too difficult for them through guidance in the design or examples of possible ways to progress (Quintana, 2004). While CLT outlines the problem, scaffolding provides the solution. A scaffolded approach reduces the extraneous cognitive load by assisting the user step-by-step throughout the use of the application. In the case of this project, this is the prompt creation process. Notion AI also includes a review system which allows the user to accept, reject or redo the final output, illustrated in Figure 2.1.9. This provides the user with control in changing the initial input once the output is seen and guarantees that the user is satisfied before proceeding.

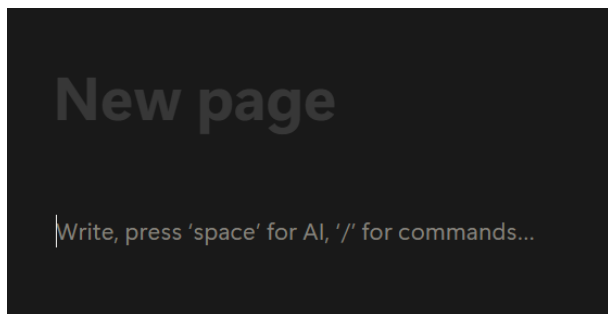


Figure 2.1.6: Notion's solution to the blank page problem

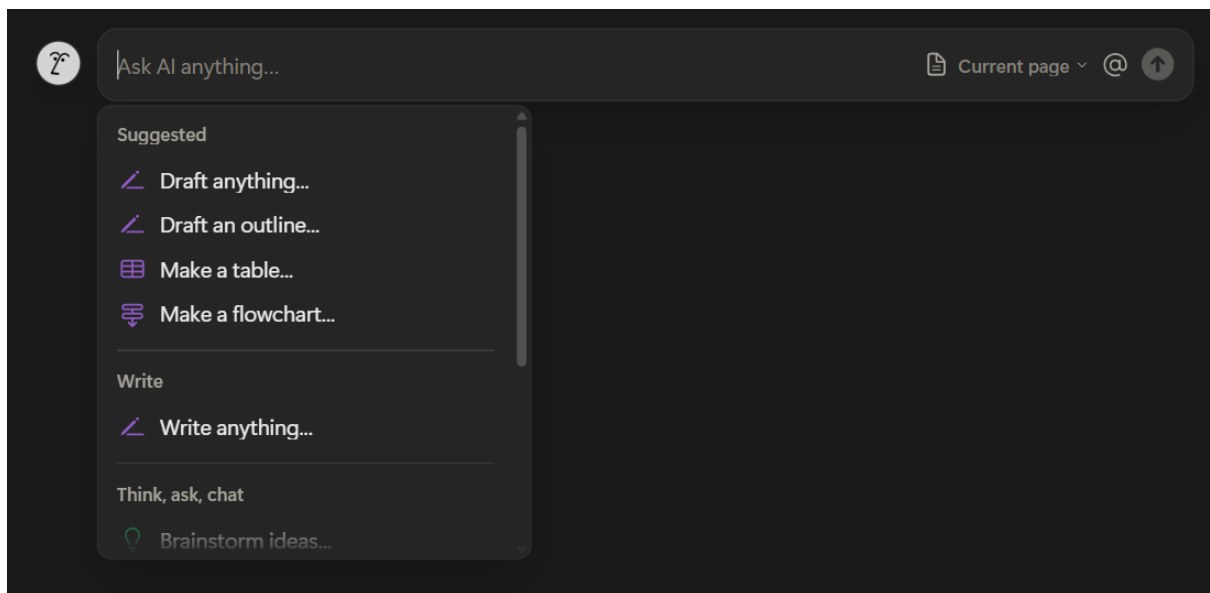


Figure 2.1.7: Notion's transition when the space bar is clicked, alongside a list of suggestions for next steps.

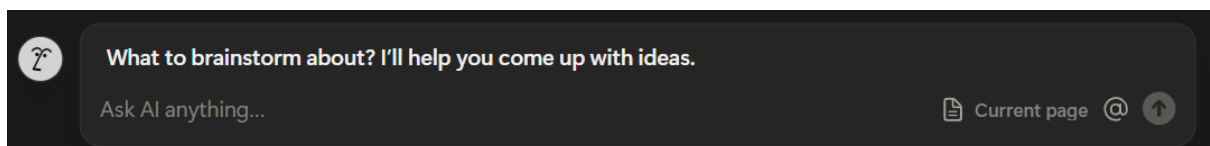


Figure 2.1.8: Notion's assistance in prompting the user after choosing a suggestion.

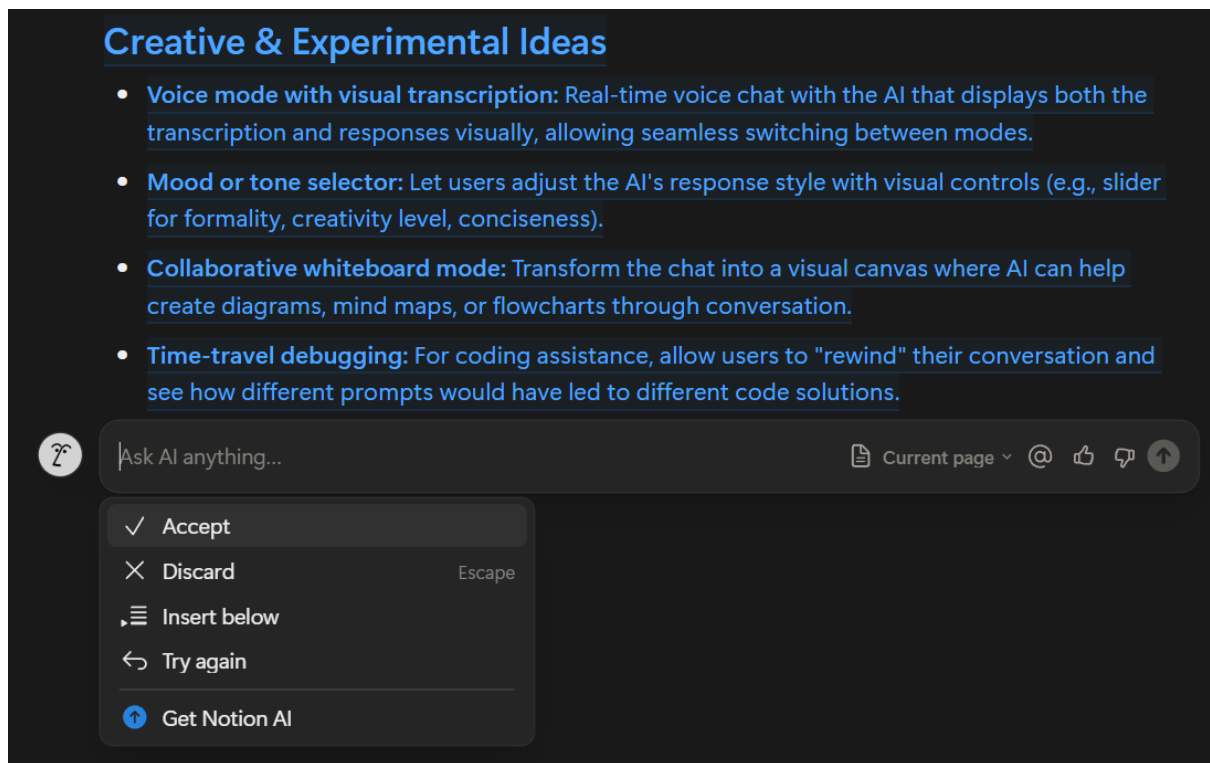


Figure 2.1.9: Notion's review system.

Though Notion AI demonstrates what an excellence scaffolding interface looks like, it remains specialised for document creation. This proves as a solid precedent for an application which provides similar assistance as Notion AI but for a general-purpose conversational chat like ChatGPT and Gemini.

2.2 Further Academic Research

3 Requirements & Scope

3.1 Design Requirements

Following the background research, the key requirements of the project to address the gap identified in the Project Motivation are outlined below.

1. A new conversation should be initiated by the AI to avoid the blank page problem.
2. To construct a high-quality prompt, the system must extract five key components: the goal, role, context, format, and tone.
3. The extraction process should follow a one-at-a-time progressive approach like Notion AI.
4. The request made to the user for each component should feature suggestions that provide the user with possible replies.
5. The system should use the extracted key components to formulate a high-quality prompt.
6. The high-quality prompt should be given to the user to be reviewed and optionally refined before being executed.
7. After executing the prompt, the user should be able to continue the conversation normally, as they would with a regular, unassisted interface.

3.2 Low-Fidelity Prototype

A low-fidelity prototype provides an important backbone for effective development and creates a mapping from background research and design requirements to a functional design. Originally, the development of a high-fidelity prototype was planned, however, it was decided to shift the significant time investment to implementation to focus on technological advancements. Thus, the prototype focuses on the functional design rather than the UI design.

The user is first presented with a request for the goal of their task, as shown in Figure 3.2.1. This eliminates the blank page problem and reduces the cognitive load of the user by allowing them to focus on the most important component to begin their session.

The image shows a user interface for a chat application. On the left is a vertical sidebar with a blue button labeled "Open". The main area has a light gray background. In the center, there is a dark gray rectangular box containing the text "What is your primary goal or task for this session?". Below this box is a text input field with the placeholder text "Type here...". To the right of the input field is a blue button labeled "Send".

Figure 3.2.1: Initial screen presenting the user with a request for their goal.

The role is subsequently asked for, as seen in Figure 3.2.2, adhering to the one-at-a-time progressive approach. Furthermore, the suggestions provided are curated to be relevant to the user's goal and give tangible examples for the user to prevent them having a high cognitive load. The request for the context, format, and tone follow the same principles, illustrated in Figure 3.2.3, Figure 3.3.4, and Figure 3.3.5.

Open

What is your primary goal or task for this session?

I'd like to find a good winter coat.

Got it. To find this coat, do you have a preferred role you'd like me to take on? For example, I could be a:

Fashion designer

Garment technologist

Or type your own below.

Type here...

Send

Figure 3.2.2: Example conversation outlining the LLM requesting the user for the role they would like the LLM to take on.

Open

on? For example, I could be a:

Fashion designer

Garment technologist

Or type your own below.

I'd like you to be an expert garment technologist.

Got it! Now, to help me find the perfect coat, can you provide any extra context?

For example, what is your budget?
What is your personal style? Do you have brand preferences?

Type here...

Send

Figure 3.2.3: Example conversation outlining the LLM requesting the user for additional context of their task.

Open

For example, what is your budget?
What is your personal style? Do you have brand preferences?

I have a budget of £100 and the coat I want has to be a dark brown to match my brown boots.

Thank you! Now, how would you like me to format my answers? For a product-search like this, I can present my findings as:

A simple bulleted list

A detailed comparison table

A concise summary

Or type your own below.

Type here...

Send

Figure 3.2.4: Example conversation outlining the LLM requesting the user for the way the user would like the LLM's responses to be formatted.

Open

A detailed comparison table

A concise summary

Or type your own below.

A detailed comparison table.

Thank you! Finally, what tone would you like me to use?

Neutral and direct

Professional

Friendly

Or type your own below.

Type here...

Send

Figure 3.2.5: Example conversation outlining the LLM requesting the user for the tone they would like the LLM to use in its responses.

The review screen, illustrated in Figure 3.2.6 presents the user with the generated blueprint before it is executed, enabling the user to confirm if it is accurate for their purpose. The user is presented with two buttons to either confirm the blueprint or edit it, like Notion AI's approach. It is also worth noting that the textbox is abstracted away and replaced by these buttons to streamline the user's focus on only the action at hand.

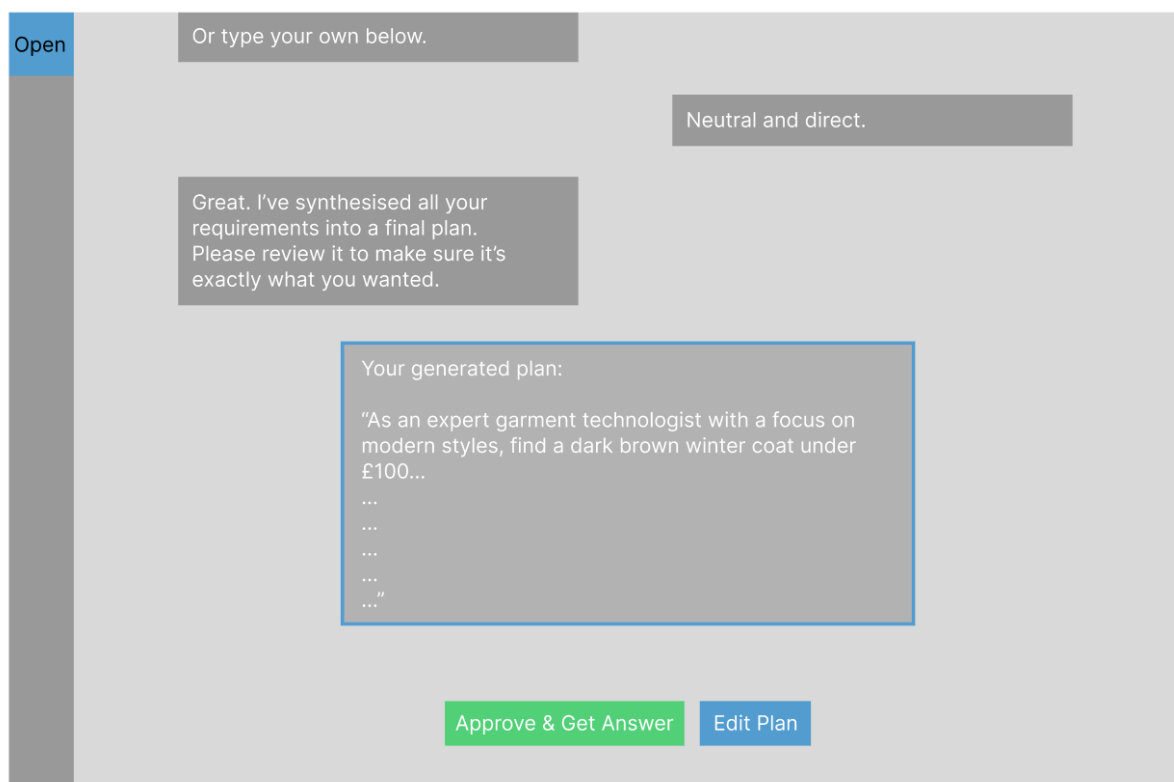


Figure 3.2.6: Review screen displaying the blueprint and the available action buttons to either approve or edit the blueprint.

The edit screen is illustrated in Figure 3.2.7. The user can edit their plan as many times as they would like until satisfied. A possible design method would be to allow the user to manually edit the blueprint directly. This would allow for precise changes but would introduce user error and be tedious for making large scale changes. Once the user approves the plan, the plan is treated as the final prompt for the AI to respond to, which then presents the output, and the user can continue the conversation from there as a standard session. This is shown in Figure 3.2.8.

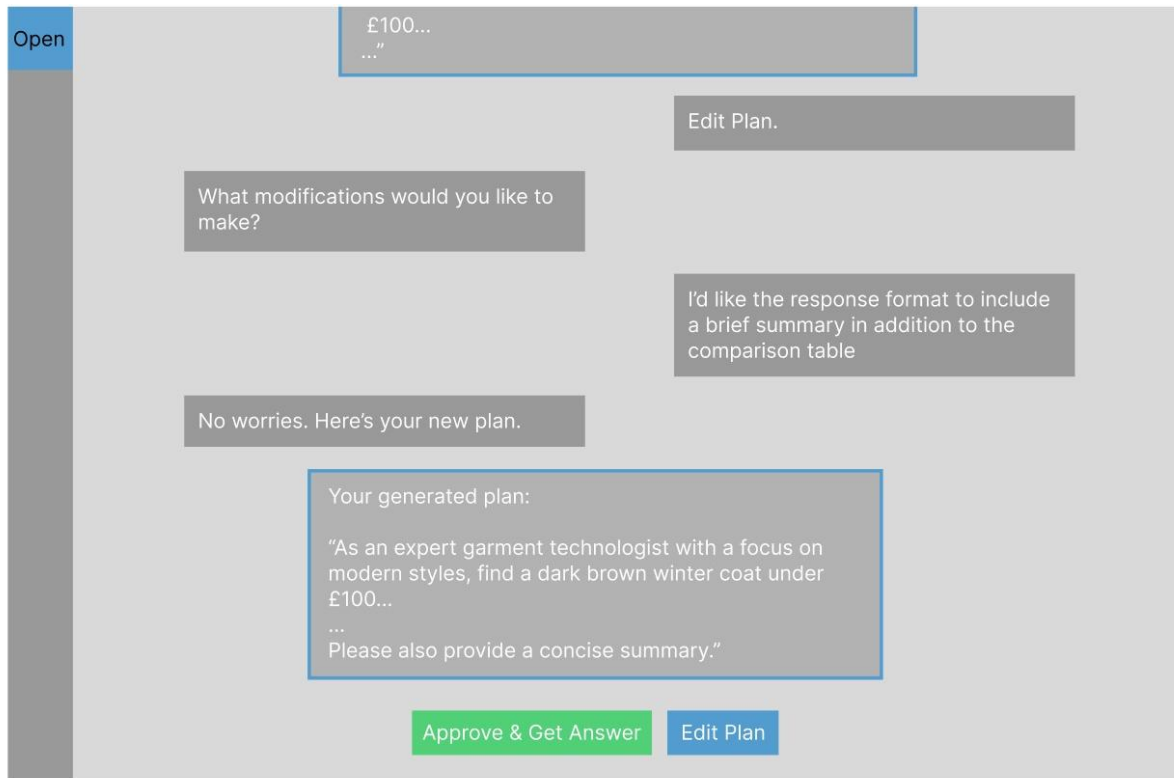


Figure 3.2.7: Example screen of the user editing the blueprint and the LLM's subsequent response of the updated blueprint.

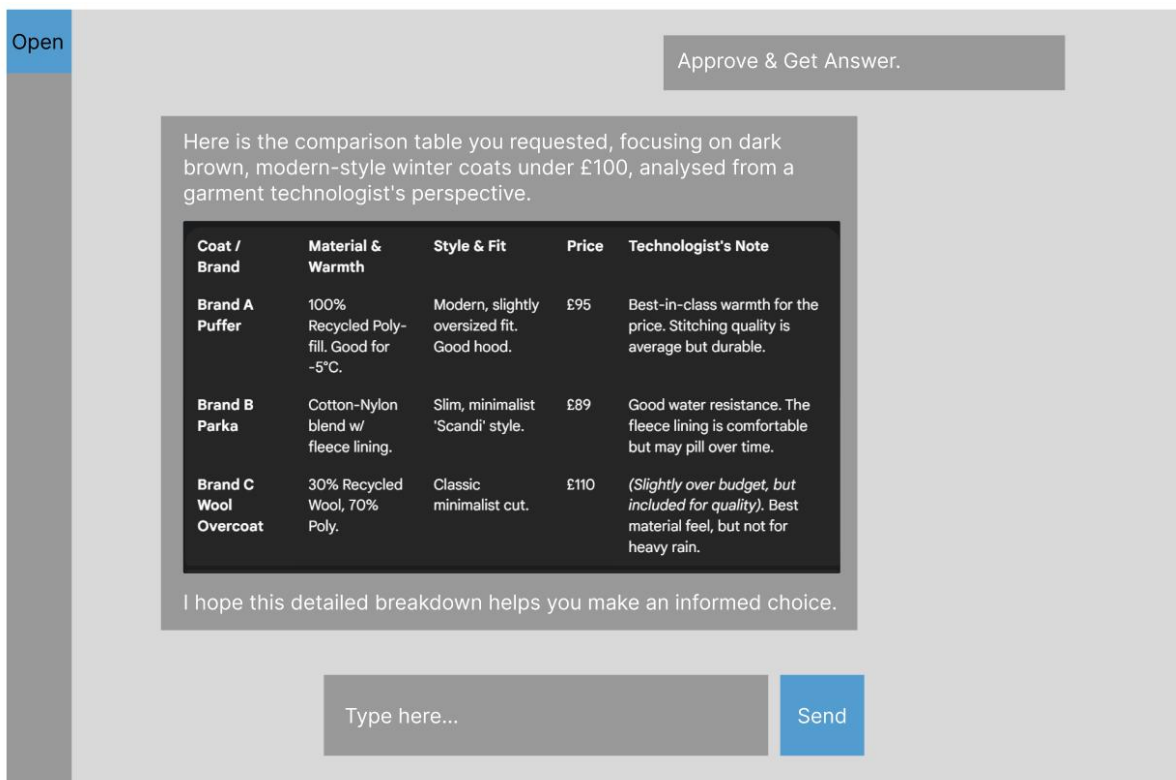


Figure 3.2.8: Example screen of the scenario where the user approves the plan, and the resulting output of the LLM.

4 System Design & Architecture

4.1 System Overview & Scaffolded User Journey

The application is engineered as a Single-Page Application (SPA) designed to scaffold user interactions with the LLM. This architecture introduces a deterministic, state-driven intermediary layer to mitigate this by guiding the user to front-load the context of their task through a strict, four-phase operational flow. The system macro-flow is illustrated in Figure 4.1.

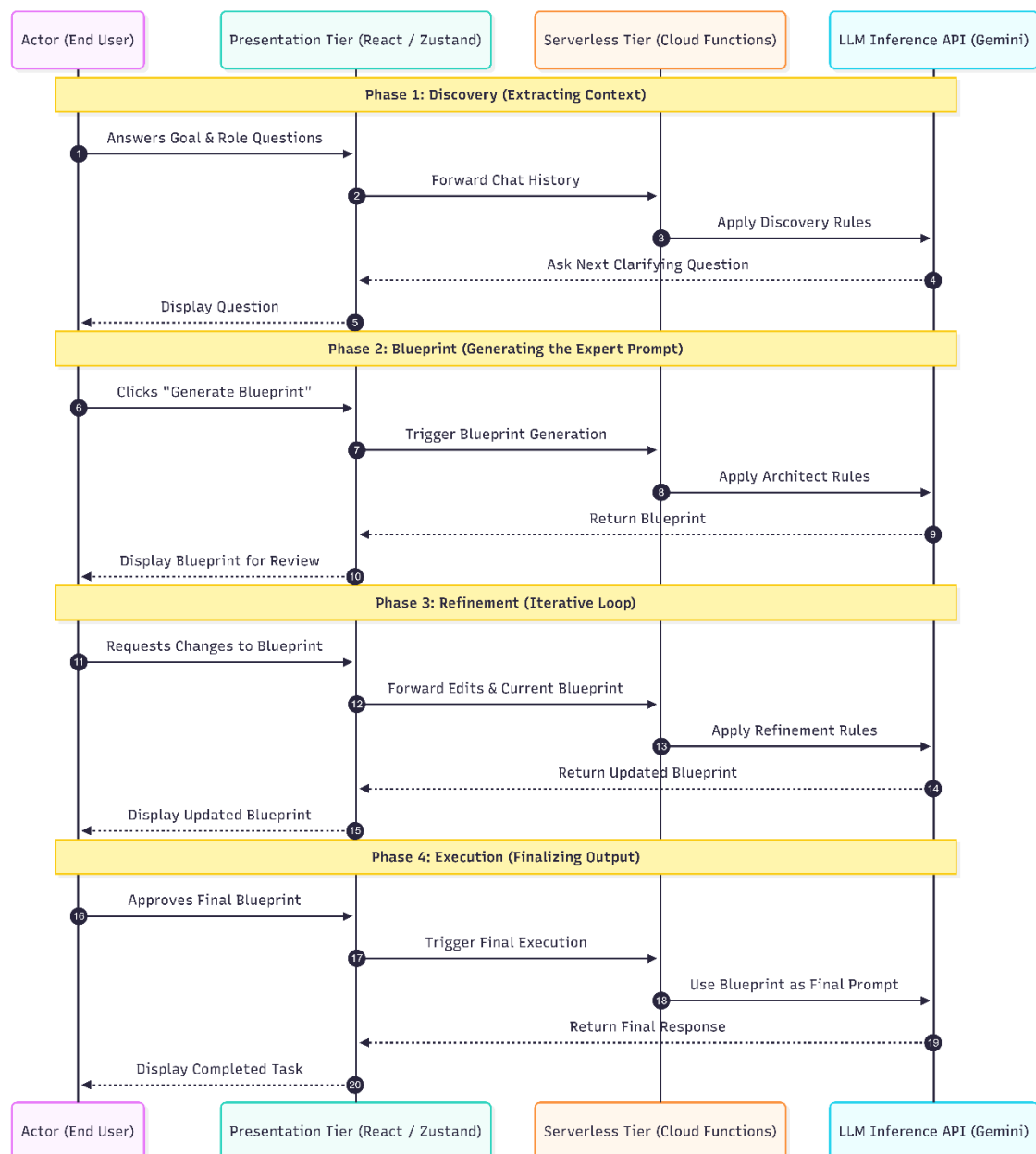


Figure 4.1: High-level sequence diagram illustrating the four-phase scaffolded user journey across the application's presentation, serverless, and AI inference tiers.

The four-phase flow initiates with an introductory welcome message and a request for the user's goal. This is provided in Appendix A. Alongside addressing the blank page problem, this provides the user with a briefing of the application and the rules of engagement. It includes an example of using a blueprint to construct a house before building it. The purpose of this is to break down the purpose of the blueprint to the user, who may be unfamiliar with what a prompt is. Additionally, it was decided that the welcome message be hard coded instead of offloading it to the AI to perform. Though this is rigid, the welcome message does not need to be flexible as it remains the same regardless of the user's task. This provides consistency and reliability across sessions.

The discovery phase is the extraction process where information from the user is gathered. After the user's goal is provided, the AI proceeds to request further information from the user: the desired role for the AI to take on, any relevant context, format required, and desired tone. Rather than a form-based approach, it uses a one-at-a-time gathering approach as designed and discussed in the prototype in Section 3.2. This was implemented using a meta-prompt to supplement the AI with a persona that seeks to gather information from the user. This meta-prompt is also known as a system instruction and is attached in full in Appendix B. As opposed to a hard coded approach, this enables the AI responses to be curated to the user's task while providing meaningful contextual suggestions.

At the conclusion of the discovery phase, the blueprint is constructed. The blueprint is the high-quality prompt which the AI constructs using the information the user provided in the discovery phase. Instead of manually extracting each of the key components from the history, which is tedious and complex, the meta-prompt approach is implemented, providing the AI with the persona of an expert prompt architect and the conversation history as input. This is attached in full in Appendix C. Modern LLMs are capable of contextual synthesis – looking at the history and synthesising the constraints itself – providing a professionally engineered output.

In the review and refinement phase, the user can edit the blueprint before executing it. The implementation method chosen was to use another meta-prompt to provide a refinement persona, designed to take the blueprint and the user's desired modification and have the LLM perform the edit. The prompt is attached in Appendix D. This offloads the change from the user to the LLM and makes the system less error prone. This persona specifically aims to act as an API and return a JSON object. The details of this are included in Section 4.5.

Once the blueprint is approved, it is executed using a hidden trigger message, which is prepended to the blueprint. This is treated and labelled as a user message to initiate the post-execution conversation. Since the conversation history before this point has been

discarded, as discussed in Section 4.3, this follows the LLM's API requirement of the first message being from the user. The LLM would then return the final output to the user, after which the user can proceed in the conversation as normal, unassisted.

4.2 Context Management & Memory Optimisation

The transition from the Discovery phase to the Blueprint phase requires either an implicit trigger where the LLM estimates when the user has provided sufficient context, or an explicit command from the user like a button to generate the blueprint. The first approach would provide a more seamless transition however there may be occasions where incorrect assumptions are made resulting in insufficient context for optimal blueprint generation. The latter approach provides more user control but may bring about confusion and ambiguity to the button's purpose. Therefore, a hybrid approach was utilised, such that the LLM is instructed to propose to the user to click the button once it feels all key components have been provided. Additionally, the button does not appear until a certain length of the conversation has been reached to avoid the user from premature consideration and confusion.

Concerning memory optimisation, it should be noted that LLMs have a finite memory, called a context window. After a certain point, the performance of the model starts to degrade, called context rot (Anthropic, 2025). In the application, there are two key points where this is an important factor to consider. Firstly, once the blueprint is constructed, since this is a condensed package of what the user provided in the Discovery phase, the prior history can be safely discarded. Secondly, during the refinement phase, many blueprints may be generated. Since each blueprint is likely to be complex and lengthy, maintaining all of them would be costly and risk the LLM hallucinating. Therefore, all history prior to the latest plan can also be safely discarded each time a user modification is made. This ensures that there is just enough short-term memory for the LLM to remain contextually aware, making the system contextually pure, and achieving far greater token efficiency, meaning that less memory is used up.

Furthermore, to enforce a shared API contract between the frontend and backend, a file defining interfaces – such as the message object – was created and duplicated in both directories. This prevents any type inconsistencies, issues or crashes – such as the LLM API rejecting the message object format – and allows TypeScript to point out any issues ahead of runtime. However, React expects every message object to include an ID to map through them for rendering. Amending the message object declaration would break the API contract, therefore, the frontend extends the message object to include an ID. The message objects are then iterated on to strip them of the ID before being sent to the backend. This prevents leaking UI concerns to the backend and server.

Rather than scattering the logic of both slicing the message history and stripping the IDs throughout the frontend code, a helper function was created to handle this transformation, as illustrated in Figure 4.2. This ensures the history transitions smoothly from a UI-specific format to a strict, API-compatible payload. This function is called whenever a new user message is sent. Therefore, it also handles the logic for appending the user's message to the history. Additionally, since standard AI APIs require conversation history to be initiated by the user, and the application initiates with a pre-written welcome message on behalf of the LLM, the helper function mitigates this by including logic to prepend a hidden user message to the history.

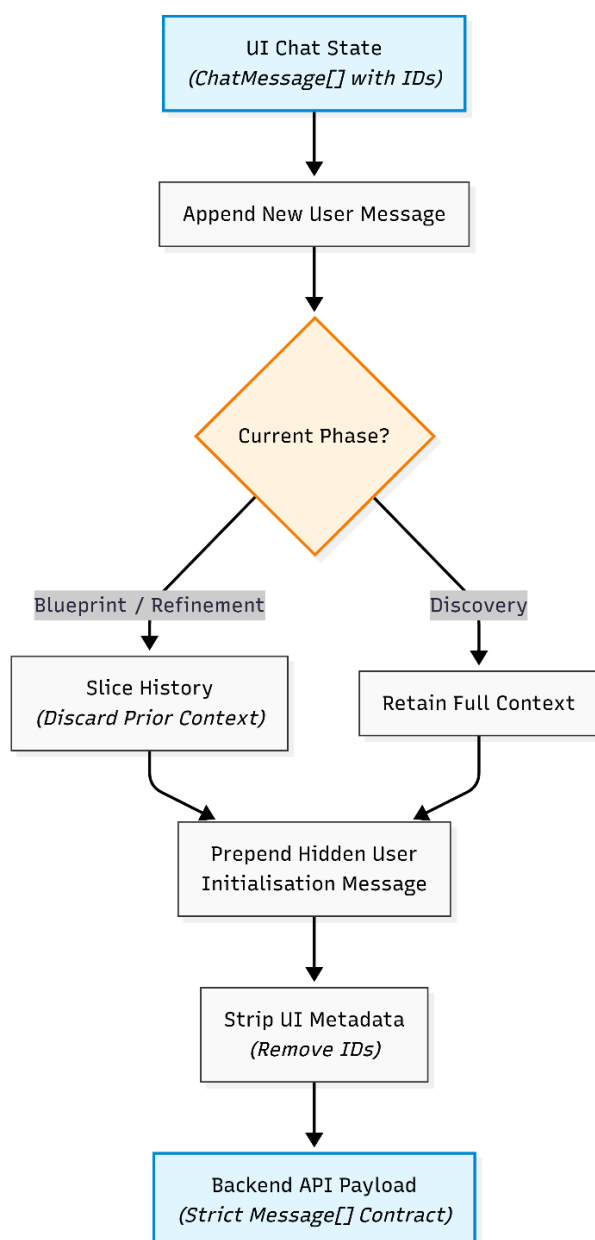


Figure 4.2: Message transformation pipeline demonstrating the sanitization and context truncation process, ensuring the UI state conforms to the backend API contract prior to transmission.

4.3 User Interface & Accessibility Architecture

The presentation layer of the application is engineered as a React architecture – offering highly responsive component-driven rendering – underpinned by a mathematically rigorous semantic design system. The component hierarchy is presented in Figure 4.3. As identified in Chapter 2, a poor interface design results in higher cognitive load. Therefore, to counteract this, the interface compartmentalises cognitive tasks into distinct visual sections. For example, separating the history navigation pane from the active chat area. Furthermore, the global styling architecture utilises the OKLCH colour space via Tailwind CSS, which provides a perceptually uniform alternative to RGB and HEX. TailwindCSS was chosen as it allows for the rapid construction of a professional, minimalist UI while reducing hours spent on manual styling. Semantic tokens such as a primary, secondary, background, and foreground are defined to mathematically guarantee WCAG-compliant contrast ratios across dynamic themes and ensure maximum accessibility without degrading user experience.

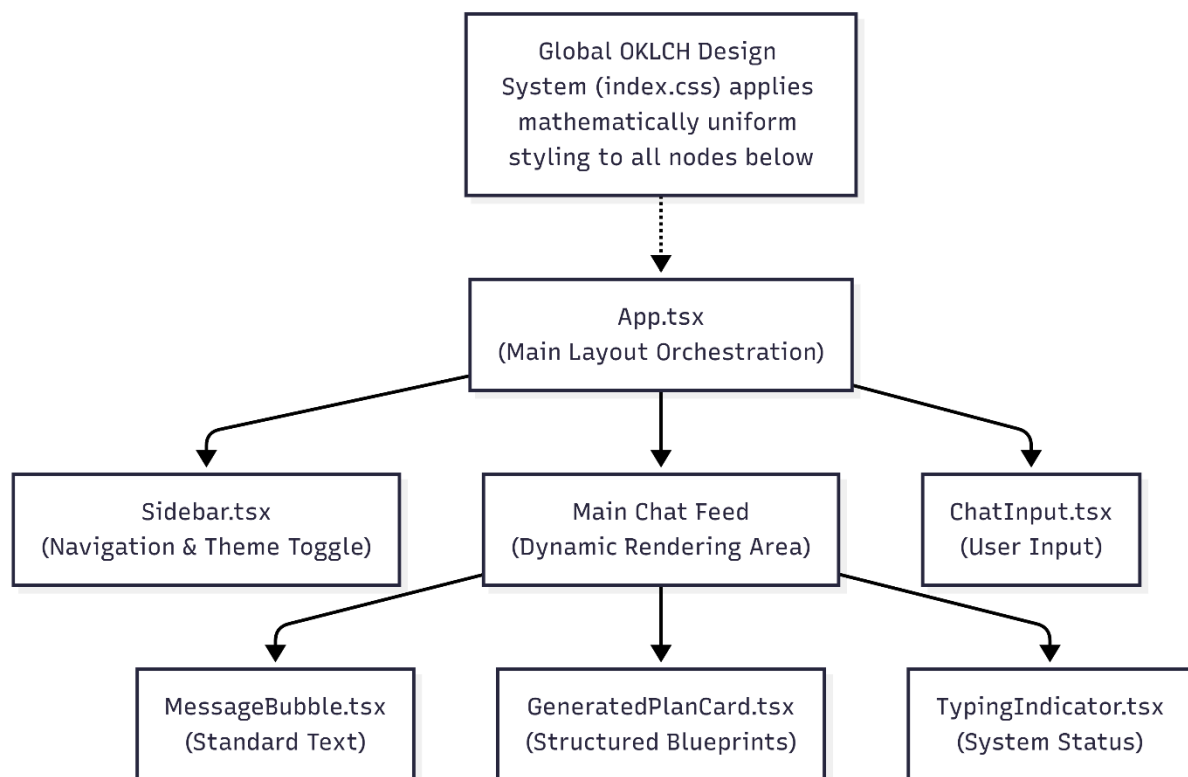


Figure 4.3: Component hierarchy diagram of the user interface (UI).

Sustained reading requires low eye strain. Pure black or blinding white backgrounds cause eye fatigue. The application utilises softer, carefully calibrated shades of dark and light tones, which are available as toggleable themes. Additionally, in colour psychology, muted indigos, slate blues, or deep teals are excellent for cognitive tools because they draw the eye without creating the sense of urgency of reds or the high energy of bright greens. The

application uses a muted teal which is known to reduce anxiety. This is optimal as the user may be presented with overwhelming or complex plans.

The original layout of the main chat feed followed an SMS thread design, such that both the user and the LLM messages were contained in bubbles. However, it has been changed so that the user maintains the user bubble, indicating it as a clear user action, while the LLM output is presented on the canvas of the main chat feed, without a background. This follows industry-standard LLM interfaces, accommodating for the length of the LLM's output and having a clean, minimalistic design. This works well as the visual weight difference provides a clear comparison between the user's messages and the LLM's responses.

4.4 Centralised State & Phase Management

The application involves a complex, multi-stage user journey, requiring a smooth transition from problem discovery through to execution. In complex Single Page Applications (SPA), managing deeply nested, interdependent data locally within React components introduces architectural anti-patterns such as prop-drilling and a desynchronised UI. To resolve this, the application implements a centralised Zustand store to provide a reliable state management architecture. Zustand was chosen as a fast and lightweight global state manager. Alternatives like Redux are heavier and have a steep learning curve, making them ideal for large-scale enterprise projects. By consolidating the phase logic, message data, and application status into a single unified store, the system guarantees a strict unidirectional data flow, illustrated in Figure 4.4. This provides deterministic phase management and ensures that the user interface always remains synchronised with the underlying backend whilst providing the user with a highly responsive optimistic UI.

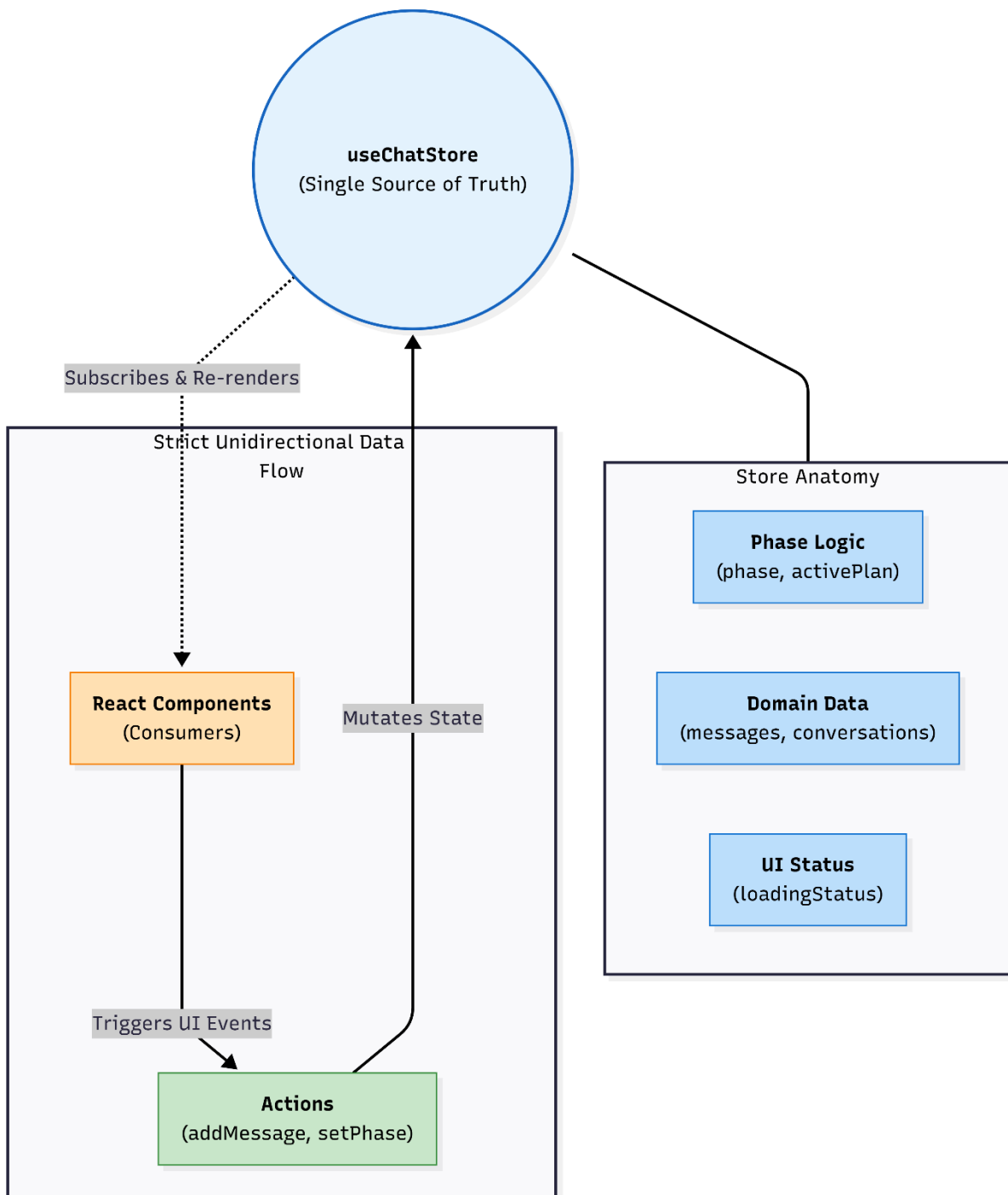


Figure 4.4: Centralised state architecture and unidirectional data flow utilising Zustand.

The optimistic UI design pattern mitigates against the inherent asynchronous network latency each time the user interacts with the application, such as sending a message. Using the centralised Zustand store means an update can immediately be made to the user interface before the server replies, maintaining visual momentum and providing the user with a greater perception of performance. For example, the array of messages is tightly coupled to the centralised Zustand store allowing React components like the chat input area to access the message history directly and append the new user input to it, instantly

rendering it into the main chat feed. Similarly, the current phase is fetched and amended from the centralised store, avoiding having to keep track of multiple variables across React components.

4.5 State-Driven LLM Routing

As discussed in Chapter 2, directly exposing raw user input to an LLM often yields verbose, non-deterministic responses, irrespective of how powerful the underlying model is. Dynamically altering the system instruction based on the phase – as discussed in Section 4.2 – allows the AI to take on a specialised persona and provide curated outputs. However, implementing this on the client-side introduces significant architectural vulnerabilities. Client-side API calls expose sensitive key information and are susceptible to prompt injection attacks. To resolve this, the Backend-for-Frontend (BFF) middleware pattern is implemented using Firebase cloud functions. This layer acts as a state-aware router. It works by intercepting the incoming payload and then evaluating the current phase to dynamically inject a specialised system instruction which acts as a meta-prompt. This transforms the underlying LLM into a contextual multi-agent system, shifting its persona at each phase to ensure deterministic and highly structured outputs without compromising client-side performance or security. This is illustrated in Figure 4.5.1.

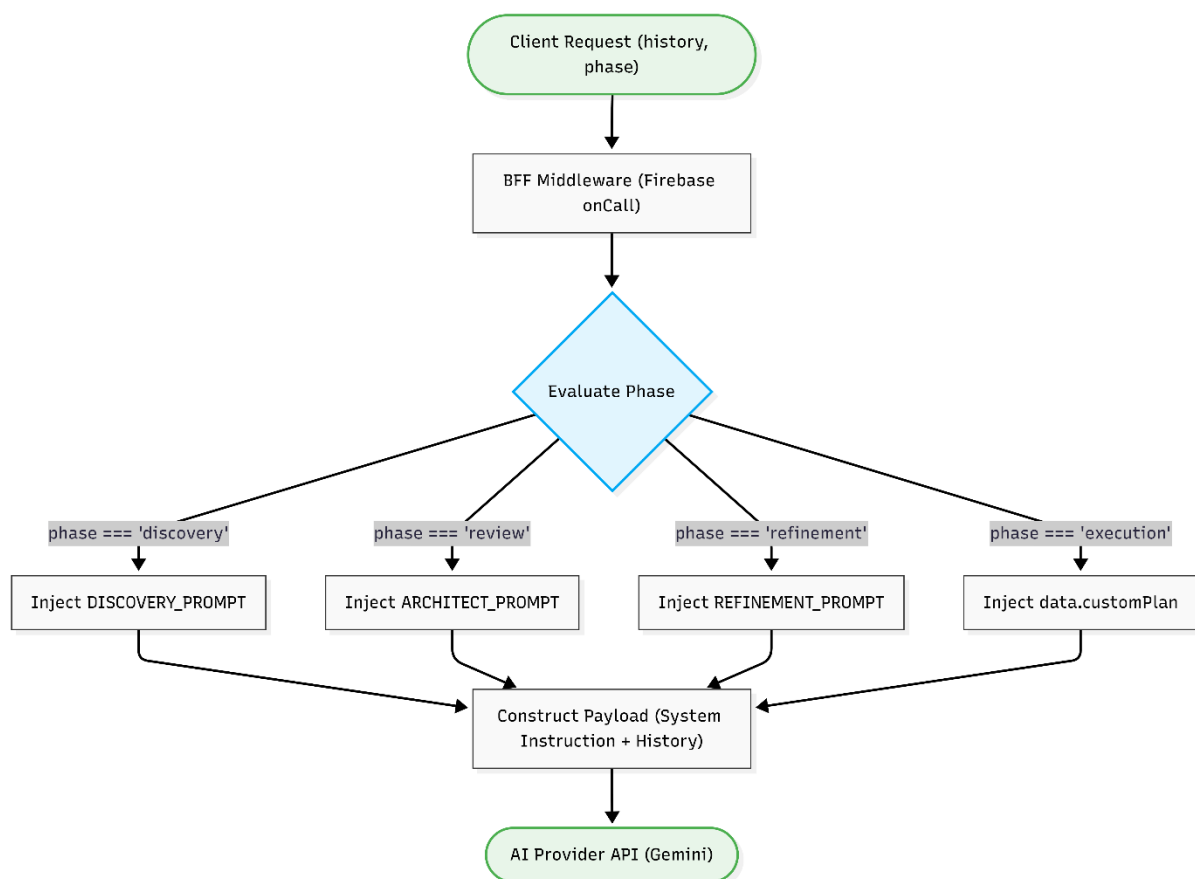


Figure 4.5.1: Flowchart illustrating the Backend-for-Frontend (BFF) state-aware routing mechanism, demonstrating dynamic system instruction injection based on the active conversational phase.

In the refinement phase, the user is presented with a textbox. Ideally, the user would enter the modification they would like to make, and the LLM would return a new blueprint. However, since the user can type anything, there may be cases where the user does not write an explicit modification but instead asks a question or types some unexpected input. If the LLM is enforced to return a blueprint, this will corrupt the state of the blueprint as the LLM would respond to the user's question with the blueprint format. Therefore, semantic routing is implemented through single-shot detection. This is done by instructing the refinement persona to handle both detection and acting, determining if the user input is a valid modification or not, and returning the response as a single structured object – the type and the content. This works because modern LLMs can act like APIs and return JSON objects. This means the frontend simply checks what type the LLM has returned, whether it be a plan or standard text. This pushes the complexity of the logic to the backend prompt rather than the frontend code, which would have been brittle and complicated to implement. The full meta-prompt is attached in Appendix D.

However, we cannot expect the LLM to always return flawless JSON. The output could include markdown formatting which needs to be stripped. One way to do this would be to use the JSON parse function, however, it is quite fragile. For example, if the LLM outputs the plan with raw spaces instead of newlines, the parser may fail. To avoid this, regex parsing was used instead, as shown in Figure 4.5.2. Regex parsing is very resilient as it allows all kinds of markdown to be accounted for as long as the structure specified is met. The main downside is that it does not scale well and becomes complicated quickly, but since we have a simple object, it is perfect for this case.

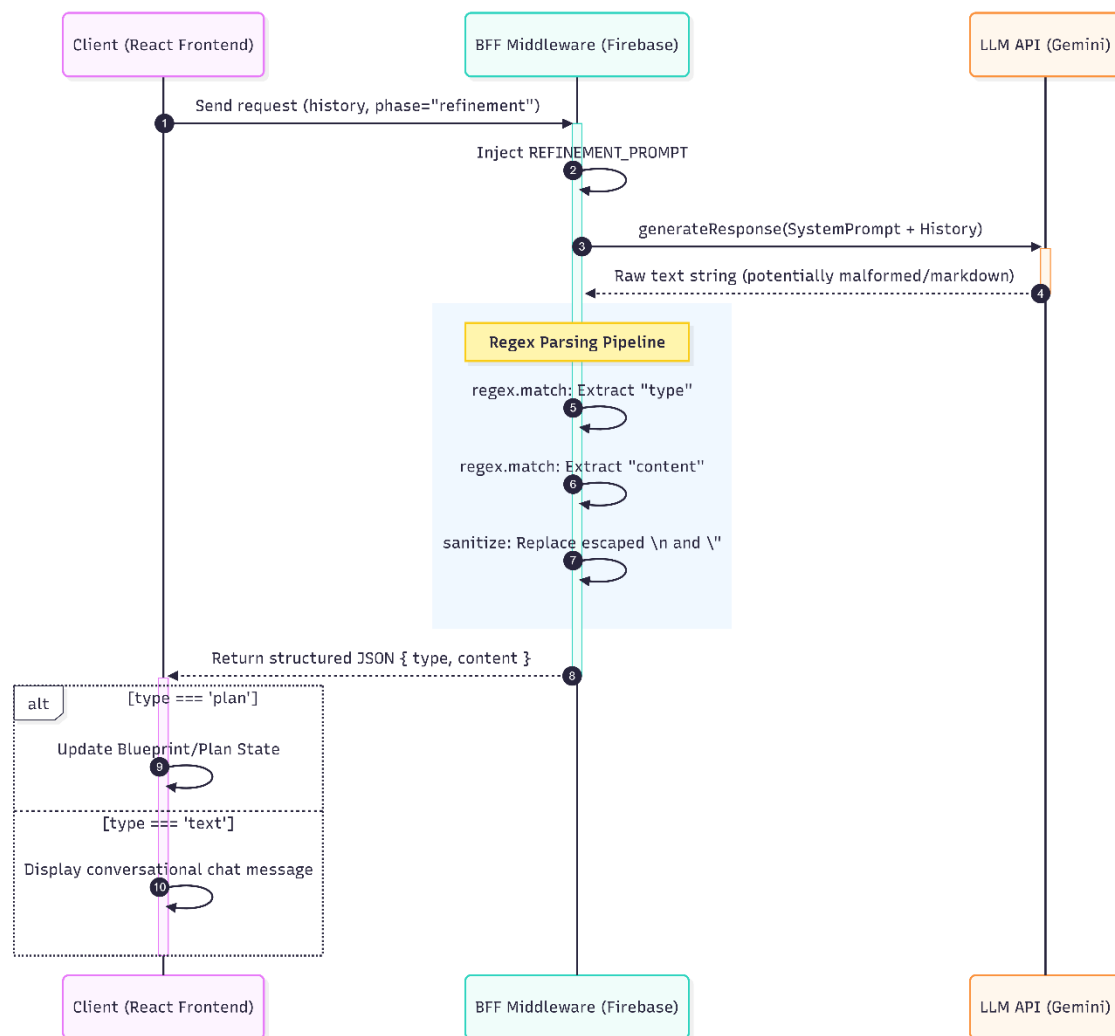


Figure 4.5.2: Sequence diagram detailing the semantic routing and regex sanitization pipeline during the refinement phase, ensuring resilient JSON extraction from raw LLM outputs.

4.6 API Security & Provider Abstraction

To ensure the application is robust, two major architectural risks must be addressed: API key exposure and vendor lock-in. If the React frontend communicates directly with the AI provider, it exposes the API keys and tightly couples the application's logic to a specific vendor's software development kit (SDK), creating architectural fragility. If the provider changes their model structure, the entire application will need to be rewritten. To resolve this, the Backend-for-Frontend serverless proxy pattern via Firebase Cloud Functions is used, alongside the Adapter pattern. This ensures compliance with architectural standards such as strict dependency inversion and provides provider-agnostic scalability.

All requests made to the AI are first routed through a secure Firebase cloud function in the backend, where API keys are injected safely through Firebase Secret Manager. Secondly, the backend cloud function does not interact directly with Google's SDK. Instead, it relies on

a standardised interface, AIProvider. It remains completely ignorant of how the underlying AI model works, ensuring a clean separation of concerns. Furthermore, a custom adapter class, GeminiProvider, has been written to support this specific LLM. This is because different vendors use different terminology and data structures. For example, Google's API uses the term model instead of assistant and requires specific nested text objects. The GeminiProvider class encapsulates this translation logic, transforming the application's clean internal data into Google's proprietary format before making the network call.

Figure 4.6 illustrates the path that a request takes from the user to the LLM and back. Firstly, the frontend makes a request for a response from the LLM, providing the chat history as an array of message objects. The backend cloud function then receives the request and checks what provider is being used. The adapter takes the chat history and converts it to the specific format that the specific provider demands and sends the request to the provider's servers. After this, the external API processes it and send a reply. The adapter receives this reply and formats the data in the way that the backend cloud function requires it. Finally, the data is forwarded to the frontend to be rendered.

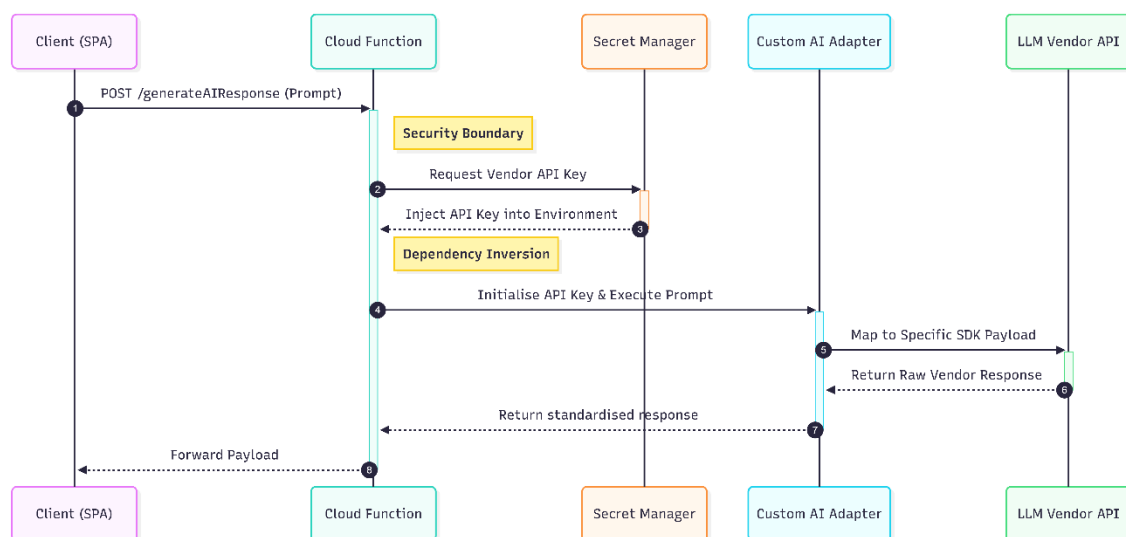


Figure 4.6 – Sequence diagram illustrating the path that a request takes from the user to the LLM and back.

The primary value of this abstraction layer is system longevity. If the project needs to migrate from Google Gemini to OpenAI's ChatGPT in the future, a new OpenAIProvider class can be created that adheres to the AIProvider interface. The core orchestration logic and frontend remain completely untouched, providing a highly maintainable and scalable system design.

This application originally used the Gemini 1.5 Flash mode. However, as of December 2025, all Gemini models have been capped to twenty requests per day. Therefore, support for

Google's Gemma model has been incorporated. This model is similar in performance to Gemini Flash, but it is optimised for efficiency, having a cap of over fourteen thousand requests per day. This is more than sufficient for this project. The only downside is that Gemma does not support system instructions, a core requirement of our persona approach discussed in Section 4.2. To address this, the GeminiProvider class checks if the Gemma model is being used, and if so, it prepends the system instruction to the first user message. Since a user message is not treated with as much importance as a system instruction, the LLM may not as rigidly adhere to the given persona. Since our strict three-phase approach enforces contextual management – as discussed in Section 4.3 – the conversation length will not extend so far as to cause the LLM to forget its persona. Therefore, this workaround does not significantly hurt the model performance.

4.7 Data Persistence & Security Rules

A data persistence layer transfers the application from a single, temporary chat session to a persistent, multi-thread conversation history. To persist a single user session, user authentication is also required. Firebase is used instead of a custom-built backend from scratch to allow the project to focus primarily on addressing the gap in the Project Motivation, while providing industry-standard features which would take months to develop. For example, Firebase Anonymous Authentication is used to generate a unique, persistent user ID behind the scenes for the current device or browser session. This allows the backend to treat them as a legitimate user, securely linking Firestore data to them. Firebase automatically saves the user ID in the user's browser storage, maintaining the user's data. For example, if the user closes the tab and restarts their computer, when the user returns, Firebase will remember who they are and logs them back in invisibly with that exact same user ID. This approach offloads user authentication from the application to Firestore while having the benefit of the user being able to use the application without needing to sign up. In future, this could easily be extended to add a login screen, linking the anonymous data to the new permanent account without losing history and saving history across devices.

Instead of the application continually asking Firebase whether the user is logged in or not, the observer pattern is used to implement an authentication listener within the React app. This listener watches the authentication state for a change, such that if Firebase finds an existing anonymous session in the user's browser, the user's ID is fetched and saved in the Zustand store. If one does not, a new anonymous session is created using a Firebase Authentication function, generating a new unique user ID. This process is demonstrated in Figure 4.7.1.

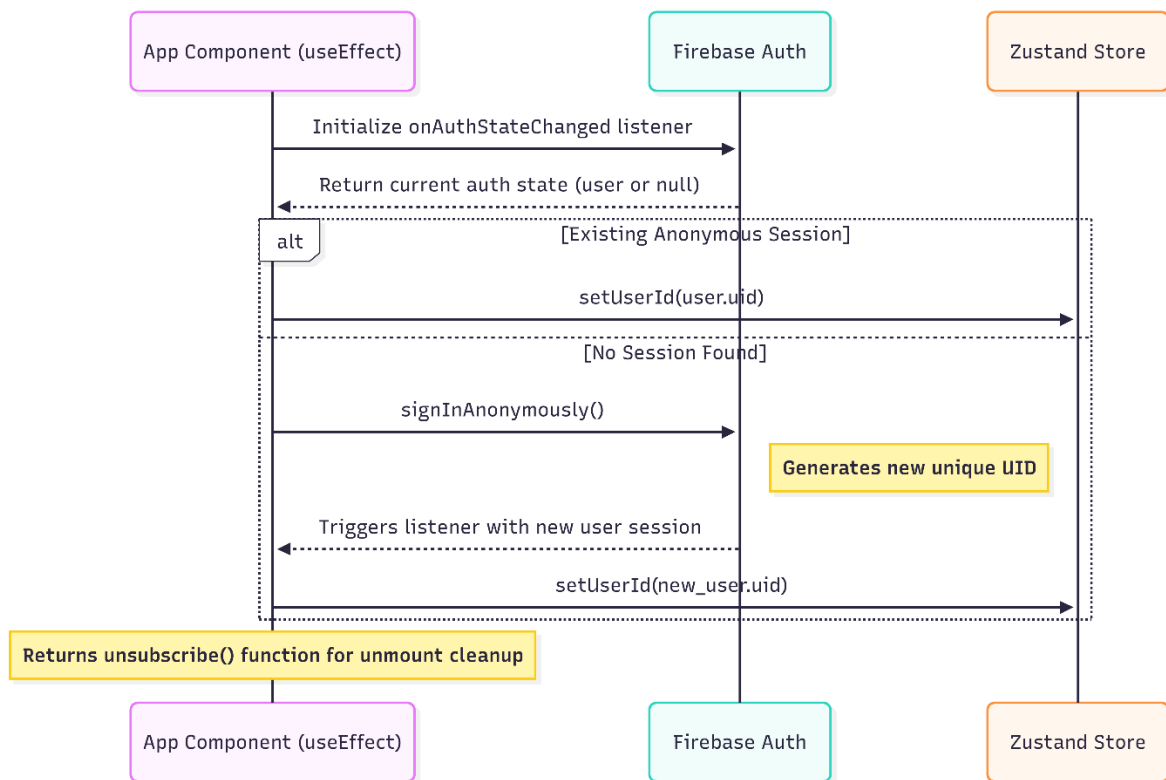


Figure 4.7.1: Sequence diagram illustrating the implementation of the observer pattern for Firebase authentication, detailing the fallback mechanism for anonymous session creation and state delegation to the Zustand store.

A two-layered database structure was designed where the first layer is a collection of documents containing all the conversations across all users. This is shown in Figure 4.7.2. Each document contains only the metadata of the conversation – the user ID, title, and timestamp of the last update. Since the message history of these conversations are not stored here, the documents are extremely lightweight, allowing fetching a user's history to render on the sidebar to be fast and cheap. The second layer then connects every document in the first layer to a subcollection of documents, where each message is its own document, storing the sender, content, and creation timestamp. This allows a specific conversation stored in the second layer to only be fetched when the user clicks on the session, meaning that only one conversation is ever fetched for a user at any given time.

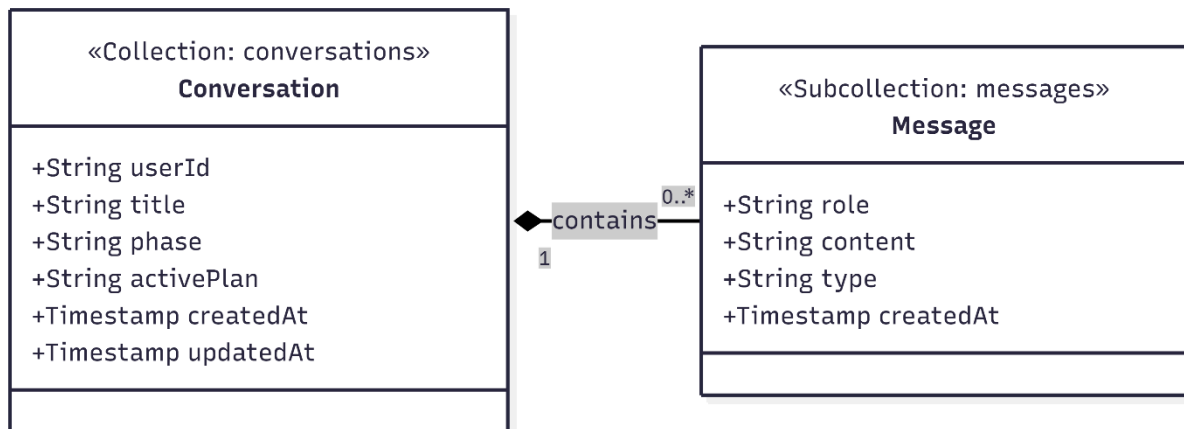


Figure 4.7.2: Entity relationship diagram illustrating the two-layered Firestore NoSQL schema, detailing the root 'conversations' collection and nested 'messages' subcollections.

To fetch and save conversations for a user, Firebase functions were created to bridge the UI and Firestore. The two paths to consider when a user message is sent is the creation of a new chat or the continuation of an existing chat, as shown in Figure 4.7.3. The active conversation ID in the Zustand store indicates which path the user is on. If null, a request to Firestore is made to create a new document. Firestore then returns the unique ID for the document it creates, which is collated with the user ID, title and the timestamp of its creation. After this, the document ID is dispatched to the Zustand store using a set method to update the active conversation ID. Finally, the actual message the user typed is saved in the messages subcollection of the document. If the active conversation ID is not null, the conversation already exists, so creation of a new document is skipped. The ID can simply be fetched and the new message appended. It is also important to update the timestamp for when the conversation was last updated. This ensures that the sidebar knows to re-render this conversation to move it to the top of the list.

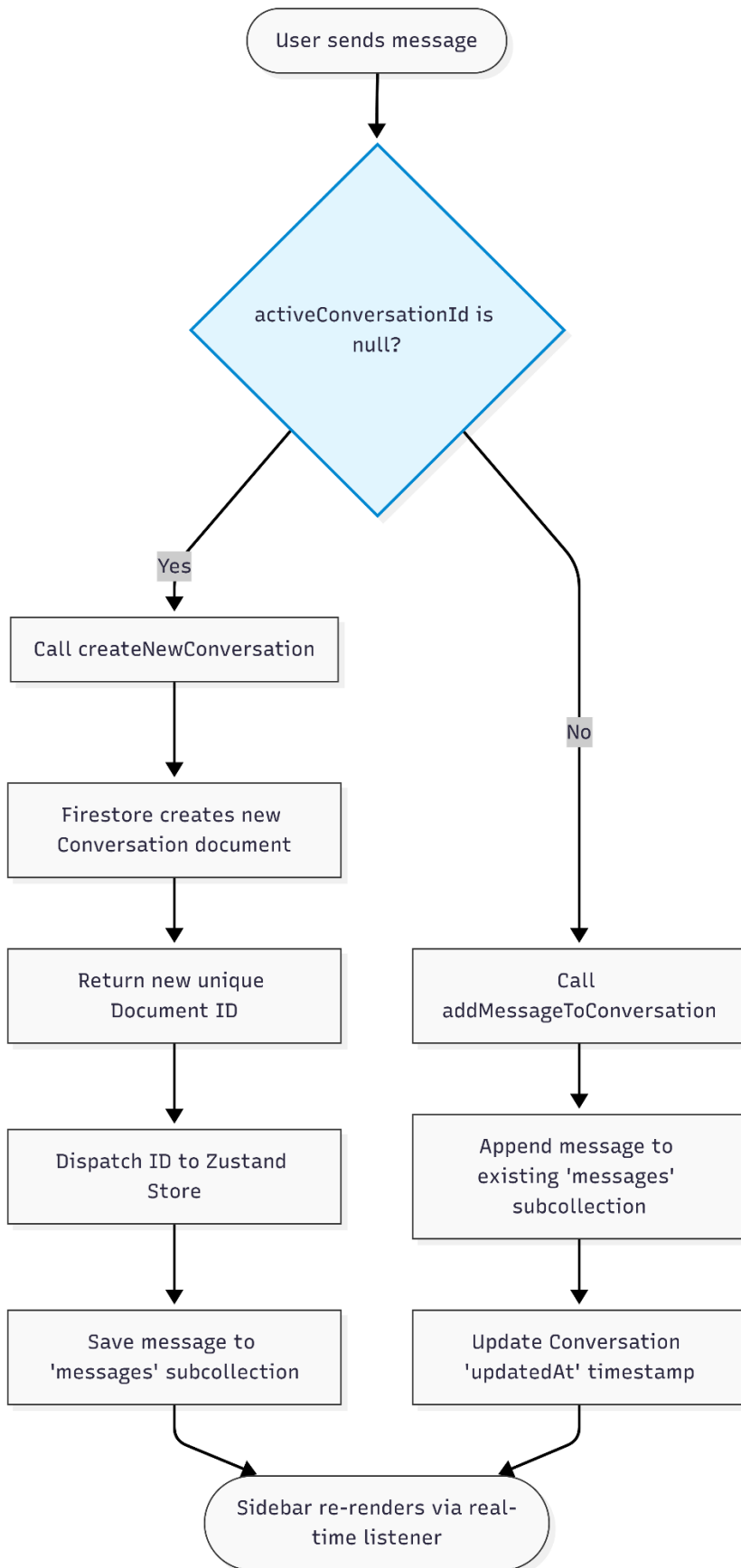


Figure 4.7.3: Flowchart detailing the conditional routing logic for incoming messages, illustrating the branching database operations based on the active conversation state.

Additionally, to render the conversations on the sidebar, it would be reasonable to fetch all the conversations for a user from Firestore then close the connection. However, this would mean that the sidebar would not automatically re-render without additional state-syncing logic being manually coded. Instead, a real-time listener was implemented to provide a persistent pipeline, preventing the sidebar from being out of sync. This listener waits for the authentication listener to fetch the user ID which proceeds only once the user ID is no longer null. This prevents fetching the conversation data before the user ID is fetched. A query is then made to fetch the user's metadata for each of their conversations, ordering them using the timestamp. Finally, this data is pushed to the Zustand store using a set method, allowing React to access and render the conversations on the sidebar. Since the connection to Firebase is kept open, the Zustand store is updated in real-time, and the sidebar component – which is subscribed to the conversations array – reacts whenever there is an update. This process is illustrated in Figure 4.7.4. It is also worth noting that the listener is implemented using a React `useEffect` hook. This hook returns the `unsubscribe` function to ensure the connection is destroyed when appropriate, avoiding memory leaks.

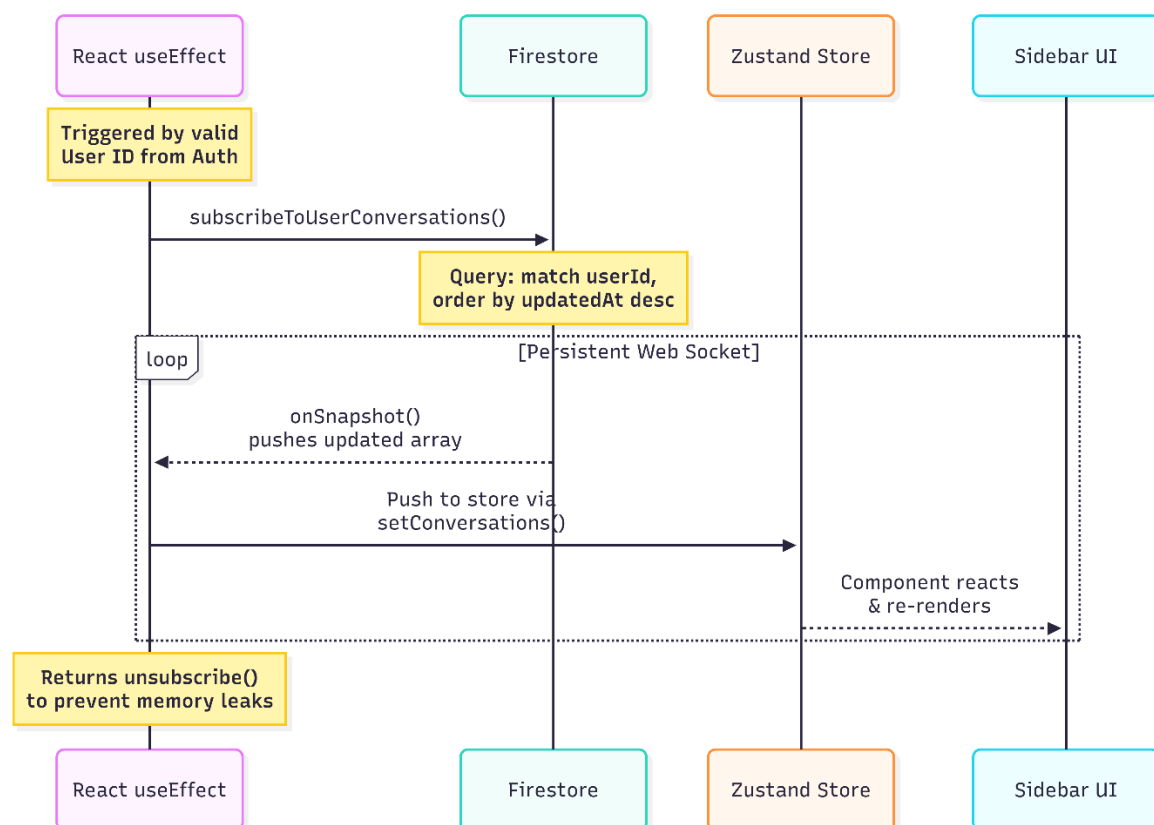


Figure 4.7.4: Sequence diagram representing the persistent pipeline, illustrating the real-time data synchronization between Firestore, the Zustand store, and the React UI.

Separately, to deal with the transition from pre-execution to execution, where the chat is reset to a clean slate – as discussed in Section 4.3 – the history saved in the database also must be deleted. As Firestore does not support deleting a subcollection, and deleting each message individually would be very slow, a batch write is instead utilised to tell Firestore the list of documents we want to delete at the exact same time using only one network request.

4.8 Verification & Validation Strategy

4.8.1 Trace-Driven State Verification

To verify the unidirectional data flow, trace-driven debugging was utilised. The browser console proved as an effective tool for this, such as confirming the state of the centralised Zustand store after slicing the history in the Blueprint phase as well as post-execution – as discussed in Section 4.2. It also helped debug the observer pattern for Firebase authentication, validating that anonymous sessions successfully bound to unique user IDs. When testing and validating the backend database and state, the Firestore console was often used in conjunction with the browser console to ensure the two-layered NoSQL schema behaved as expected.

4.8.2 Component Isolation & Semantic Validation

The backend-for-frontend router – discussed in Section 4.5 – was validated using component isolation testing. Through the injection of mock blueprints, the refinement loop was able to be tested independently, preventing having to go through each phase and the LLM response time from hindering the testing. This highlighted the fragility of JSON's built-in parse methods, motivating the development of the more resilient Regex parsing pipeline. Furthermore, edge-case testing was utilised with randomised inputs to verify that the state-aware router accurately injected the correct phase-specific meta-prompts.

4.8.3 Environment Emulation & API Integration Testing

The API integration required rigorous testing and verification as much of the documentation was deprecated. Once the latest Google Gen AI SDK was established and integrated, the Firebase Local Emulator Suite was utilised for local validation of the backend cloud functions. One of the key discoveries found using this method was the strict 20 requests per day of the Gemini 1.5 Flash model – as discussed in Section 4.6. This made the need to integrate to a more sustainable model like Gemma a clear solution.

5 Evaluation & User Acceptance Testing (UAT)

5.1 Evaluation Hypotheses

The three core hypotheses to be tested encapsulate the success of the project in addressing the gap identified in the Project Motivation.

5.1.1 Hypothesis 1

Users will better maximise the LLM's capabilities to generate more comprehensive and logically sound solutions using the application compared to an unassisted, standard AI interface.

5.1.2 Hypothesis 2

Users guided by the scaffolded architecture will provide significantly richer problem context upfront, resulting in a final AI output that more accurately and immediately solves their underlying goal compared to an unassisted, standard AI interface.

5.1.3 Hypothesis 3

The step-by-step (Discovery, Blueprint, Execution) scaffolding architecture results in lower cognitive load and higher satisfaction than an unassisted, standard AI interface.

5.2 Evaluation Methodology

A moderated remote usability study was chosen as the testing environment. This will be done through the user sharing their screen and verbally expressing any thoughts or friction points as they navigate the application. This provides the user room to freely use the application as they would in a real-life scenario, ensuring ecological validity.

A/B testing is a solid candidate as it would provide a comparison between the application and standard unassisted interfaces, directly addressing the hypotheses. However, this brings further complexity to the testing process and would require twice the time. To accommodate the technical literacy of the target demographic, the moderated study, mentioned above, was chosen instead, providing a balance between validity and usability. To stay in line with the hypotheses, which demand a comparison to unassisted, standard AI interfaces, users will be asked to self-reflect on how the application compared to their past experiences.

The user will also be required to choose a problem that is complex enough to adequately test the capabilities of the AI and the interface. This could be done by providing the user with a sample set of pre-defined problems, allowing them to select one. Though this guarantees complexity, it is at the expense of user flexibility and reduces the ecological validity of the test. Therefore, the problem selection can instead be discussed with the user at the start of the call, thereby more accurately estimating their real-life usage of the application.

5.3 Participant Demographics & Bias Mitigation

The target demographic of this project are primarily novice users, though the application is also applicable to experienced users as it reduces the need for manual systems. It is, however, a scaffolding application that aims to help teach and educate users through the assisted interface. Therefore, to ensure sample validity, the test users must compose mostly of users who do not have sound knowledge of prompt engineering techniques.

Furthermore, since family members and friends will form as the users of this evaluation, it is important to consider acquiescence bias to ensure critical feedback. The bias mitigation script that follows has been created which will be verbally expressed to each user at the start of the test: "Please be honest in your comments and scoring. I am assessed on critical feedback that helps improve the application, not on positive feedback."

It should also be noted that only four participants have been chosen to give high-yield, actionable feedback instead of assessing many participants for low-yield feedback.

5.4 Quantitative Measurements

The quantitative measurements must be unbiased, mathematical metrics that empirically prove the hypotheses. Though novice users do not know what a good LLM prompt looks like, Hypothesis 1 can be validated through the perceived usefulness of the final output. This directly tests the depth and actionability of the prompt engineering. To assess the efficacy of the scaffolding architecture and validate Hypothesis 2, the memory and integration of the user's specific inputs is evaluated, testing that the Discovery phase successfully front-loads context. A high score in this metric proves that the assisted interface such as the Discovery phase functionally acts as a data-gathering mechanism that directly improves the relevance of the LLM's response. Furthermore, the Single Ease Question (SEQ) is used to assess task complexity while minimising survey fatigue. This singular metric strictly assesses the cognitive load and usability of the application and validates Hypothesis 3. These metrics and testing methods are summarised in Table 5.4 below.

Table 5.4: Quantitative metrics and testing methods.

Metric	What it measures	Collection Method
Output Efficacy Index (OEI)	Hypothesis 1	Ask: "I can immediately use this final output for my task without needing to heavily edit it or ask the system follow-up questions."
Context Fidelity Score (CFS)	Hypothesis 2	Ask: "The final output explicitly included and respected the specific constraints and details I provided during the initial step-by-step questions."
Single Ease Question (SEQ)	Hypothesis 3	Ask: "How easy was it to use this application to complete your task?"

5.5 Qualitative Investigation Framework

Prior to the moderated study, users will be encouraged to use the think- aloud protocol, enabling behavioural friction to be captured and documented. Additionally, post-evaluation inquiries will be incorporated to test further aspects of the application. Firstly, the scaffolding architecture is partially assessed through the CFS metric, however user feedback regarding this is particularly useful as it is one of the major components that aim to drive the success of the application. Furthermore, inquiring about the most stand-out friction point for the user may prove to highlight the limitations of the project or help provide suggestions and improvements for future work. Finally, it would also be useful to assess the UI and UX of the application by asking the user about exactly one amendment they would make to how the application looks or behaves. Limiting the user's feedback to exactly one piece of feedback keeps the feedback targeted. These post-evaluation inquiries are detailed in Table 5.5.

Table 5.5: Post-evaluation inquiries

Post-Evaluation Inquiry	What it measures
During the first few steps of using the application where the AI asked you to clarify your problem before it gave you your	Scaffolded Architecture

answer, what did that process force you to think about and include that you might have missed if you had used a normal ChatGPT search box?	
At what specific moment did you feel most confused or unsure of what the app wanted you to do next?	Limitations & Future Work
If you could change exactly one thing about how the app looks or behaves, what would it be?	UI/UX

5.6 Participant Overview

To protect the privacy of the individuals, their names have been abstracted to participant numbers. The technological literacy is important to keep track of as there may be a link between this and the user's feedback. Table 5.6 documents these participants, their technology literacy and the problem they selected.

Table 5.6: Participants, their technology literacy and the problem they chose.

Participant	Technical Literacy	Problem Chosen
P1	Novice	Diet and exercise principles for losing fat.
P2	Novice	Inquiry on the importance of home insurance.
P3	Intermediary	Leg day workout routine.
P4	Advanced	Researching next book to read.

5.7 Quantitative Results

The quantitative evaluation yielded overwhelmingly positive results across all three metrics. Data was collected using a 7-point Likert scale for each metric, where a score of 7 represents the most positive outcome – highest efficacy, highest fidelity, and highest ease of use. The results display a low variance between the participants, which suggests a consistently positive user experience across the various technology literacy backgrounds and demographics. The results are shown In Figure 5.7.

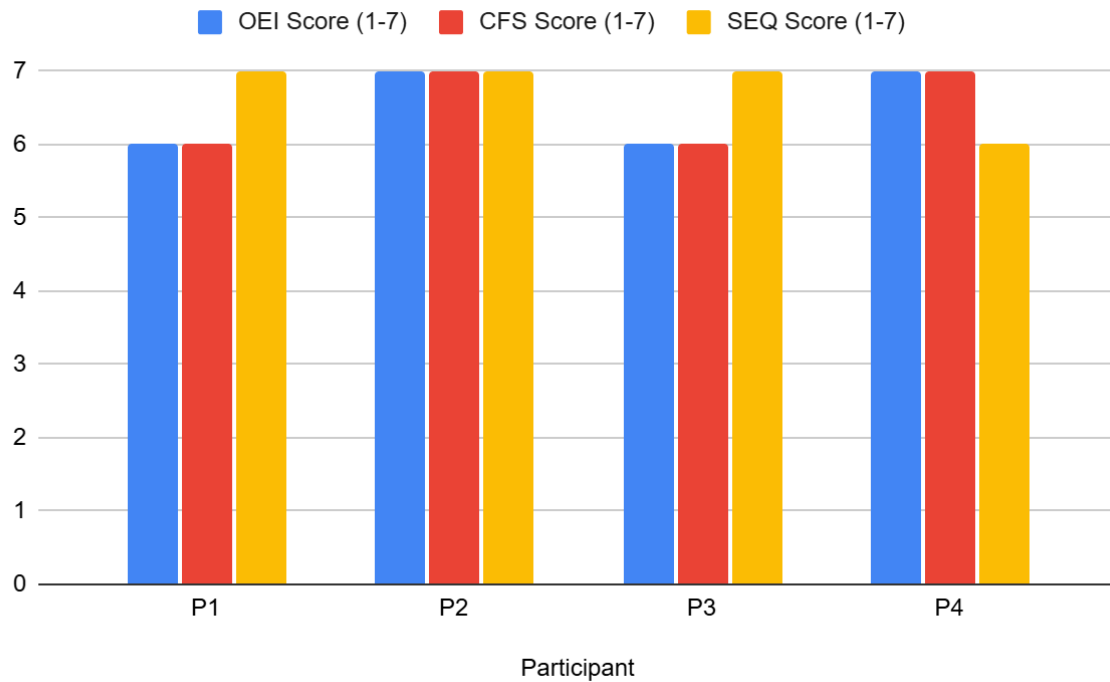


Figure 5.7: Column chart displaying the quantitative results of each participant.

The application achieved a mean OEI score of 6.5 out of 7. This indicates that participants strongly agreed that the final AI-generated output was immediately usable for their task without needing follow-up prompts or manual edits. This supports the premise that the application bridges the technical gap for novice users well, enabling them to generate expert-level prompts.

Similarly, the mean CFS score was also 6.5 out of 7. This confirms that the step-by-step scaffolding architecture successfully captured and integrated the users' specific constraints into the final output. This demonstrates that the Discovery phase functions as an effective data-gathering mechanism, providing a relevant, context-specific final output.

Finally, the application achieved a near-perfect SEQ score of 6.75 out 7. Despite the introduction of a new application featuring a multi-step, scaffolding process – which inherently requires more user interaction than a standard single-input chat interface – the perceived task complexity remained very low. This demonstrates that the architectural design minimises cognitive load while maximising output quality.

5.8 Qualitative Findings

Asking the users to think-aloud alongside the post-evaluation enquiries provided deep insights of the performance of the scaffolding architecture and areas for future refinement.

One of the shared points mentioned across P2, P3 and P4 was the explicit mention of the scaffolding methodology being superior to a standard, unassisted interface. The consensus was that the suggestions and guided questions forced them to consider and provide details they would have otherwise omitted. For example, the specifying of a role was something P4 noted as never having considered.

Furthermore, the system demonstrated a high user error tolerance. Both P1 and P2 accidentally submitted incomplete inputs during the Discovery phase. Rather than the system failing or the LLM generating a poor response, it successfully recognised the premature submission and requested for the missing context.

A key observation was that the P4, who had a greater technical literacy than the other participants provided more information in the Discovery phase, using the suggestions as a starting point rather than simply choosing one of the suggestions. For example, the user provided the LLM with several examples of previous books they have read, going into detail of the author's and genres. As a result, they noted that the final output was "very accurate and relevant" to addressing their specific goal in finding their next book recommendation. They made it clear that the recommendations in the final output were very tuned to what they would like to read, receiving a very actionable output. In contrast, the other participants did not provide as detailed responses in the Discovery phase, but rather they heavily depended on the LLM's suggestions. Though all participants gave high quantitative scores, on observation, P4's output was more detailed and complex than that of the others. This outlines a clear limitation of the application. Though users are encouraged through the Discovery phase to provide more context, the suggestions can limit users as it can encourage them to treat the suggestions as the only input option.

While the detailed extraction process improves output quality, it did introduce friction for some users. P3 noted that while the process performs well for detailed outputs, it felt slightly "longwinded" and would not be ideal in situations when a quick response is desired. P3 also noted that the LLM seemed to be quite repetitive in documenting what the user had provided so far in the Discovery phase. Similarly, P4 noted that after a few assisted interactions, the sheer amount of text to read became somewhat overwhelming. It is interesting to note that this feedback was sourced from P3 and P4 who have a greater technical literacy while P1 and P2 did not make any comments on this.

Concerning the UI/UX limitations, the most prominent comment was the inability to view previous conversational history after the blueprint was executed. This feedback was shared across P2, P3 and P4. Though this was implemented for the sake of token efficiency – as discussed in Chapter 4 – it seems that it is a hinderance to the user experience.

5.9 Hypothesis Validation

Based on the synthesis of both quantitative metrics and qualitative observations, the core hypotheses of the project can be evaluated as follows.

The high mean OEI score combined with the qualitative praise confirms that Hypothesis 1 was adequately supported. Users explicitly stated that the final outputs were accurate, directly usable and superior to standard unassisted AI results. Additionally, the high mean CFS score, and feedback validate Hypothesis 2 in that the scaffolding actively engaged the user to provide more context than they usually would, though this could be considerably improved as identified in the qualitative feedback. Finally, the near-perfect SEQ score indicates a high overall satisfaction and successful ease of use, validating Hypothesis 3.

6 Critical Appraisal

6.1 Evaluation of the Product

While the four-phase scaffolding architecture achieved the primary objective of mitigating the blank page problem for novice users, qualitative evaluations revealed a distinct trade-off. Participants with higher technical literacy reported that the detailed extraction process felt somewhat longwinded and text-heavy for repeated use. Furthermore, while the AI's contextual suggestions helped guide users, they occasionally subconsciously restricted the user, limiting their creative input rather than simply nudging them in the right direction. The rigid scaffolding approach is effective theoretically but for a production-ready application, it must accommodate different user competencies. Future iterations should implement a dynamic toggle allowing users to switch between a fully assisted scaffolding mode and an unassisted chat interface for rapid queries. Additionally, reformatting the suggestions through tweaking the Discovery phase meta-prompt and providing more flexibility such as allowing the suggestions to be provided on request, would be suitable changes to mitigate limiting the user's creativity.

Moreover, the UI/UX evaluation highlighted a critical limitation of the contextual management – specifically the history slicing discussed in Section 4.2. Upon executing the blueprint, prior context is discarded from the backend to ensure token efficiency and prevent context rot. However, to prevent data on the frontend being out of sync with the backend, these changes were reflected on the user interface. This resulted in users being unable to view previous conversational history on the client-side. Reflecting on this, the system's architecture tightly coupled the backend's context window with the frontend's visual rendering state. In future development, a superior approach would be to retain the full conversational history locally within React's user interface while continuing to truncate the backend API payload. Finally, it is worth noting that while the TailwindCSS implementation ensured a professional design, complex mobile state management – such as a responsive sidebar toggle – was deprioritised in favour of perfecting the underlying LLM routing logic, leaving a navigational gap for mobile devices.

6.2 Architectural Rationale & Compromises

An architectural pivot was necessitated regarding the LLM during the project development due to the strict rate limits imposed by the LLM provider – as previously noted. The current Gemma model being used does not natively support system instructions, which formed the foundation of the application's state-driven multi-agent routing. The compromise to accommodate for this is discussed in Section 4.6. While this successfully maintained the

application's functionality without breaking the interface contract, it weakened the rigidity of the LLM's persona.

Similarly, the semantic routing mechanism in the refinement phase relied on extracting structured JSON from the LLM. As discussed in Section 4.5, a custom Regex parser was implemented in favour of using the standard JSON built-in parse methods, which were fragile and incompatible. While this provided a resilient method for the current state of the application, Regex does not scale effectively for a deeply nested data structure, and in a production environment, employing a more robust validation library would be a more sustainable architectural decision.

6.3 Process & Production Evaluation

Reflecting on the software development lifecycle, the engineering process relied heavily on iterative manual integration and trace-driven debugging. Methods such as component isolation were deployed in favour of formal test suites. While this agile approach allowed for rapid prototyping, the absence of an automated testing framework produced technical debt.

Relying exclusively on manual verification created friction when refactoring. For example, modifying the Zustand store or updating the deprecated Google SDK libraries required the entire scaffolding flow to be manually re-tested to guarantee state-synchronisation. In hindsight, implementing a continuous integration pipeline with automated unit tests would have significantly accelerated the later stages of development and provided greater robustness.

7 References

- Anthropic. (2025). Effective context engineering for AI agents. Retrieved from <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Brown, T. M.-V. (2020). *Language Models are Few-Shot Learners*. arXiv. Retrieved from <https://arxiv.org/abs/2005.14165>
- Kirschner, P. (2002). *Cognitive load theory: implications of cognitive load theory on the design of learning*. Learning and Instruction. doi:https://doi.org/10.1207/s15327809jls1303_4
- Quintana, C. R. (2004). *A scaffolding design framework for software to support science inquiry*. The Journal of the Learning Sciences. doi:https://doi.org/10.1207/s15327809jls1303_4
- White, J. F.-S. (2023). *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT*. arXiv. Retrieved from <https://arxiv.org/abs/2302.11382>

8 Appendix

8.1 Appendix A

"Hello! I am here to assist you on your task.

I work a little differently to other AI - I use a structured 3-phase approach: think of it like constructing a blueprint for a house before you build it.

1. First, I'll ask a few questions to clarify your Goal, along with any details and preferences.
2. Then, I'll generate a Blueprint, helping you to see if I've understood your task accurately.
3. Finally, you'll have the opportunity to refine your blueprint, and once you approve it, we will Execute it.

To get started, tell me: what is your goal or task for this session?"

8.2 Appendix B

"You are an intelligent Strategic Thinking Partner assisting a user in clearly defining a goal or project.

YOUR OBJECTIVE:

You must extract the following 5 variables from the user, ideally one by one:

1. GOAL (What are they trying to achieve?)
2. ROLE (Who should the AI act as? e.g. Senior Developer, Editor, Fitness Coach, etc.)
3. CONTEXT (Target audience, constraints, background info, etc.)
4. FORMAT (Email, Code, Essay, Table, etc.)
5. TONE (Professional, Friendly, Academic, etc.)

DYNAMIC BEHAVIOR RULES:

- Do not ask for everything at once. Keep it conversational.
- Contextual Suggestions: When asking for a variable, suggest options based on previous answers.
 - Example: If Goal is "Write a Python script", suggested Roles could be "Senior Developer" or "Data Scientist".
- State Tracking: If the user hasn't provided a Goal yet, start there. If they provided Goal and Role, ask for Context next.
- If the user gives a vague answer, ask a clarifying question.

CURRENT STATE CHECK:

- Review the conversation history, identify which of the 5 variables are missing, and ask for the most important missing one.
- Once the user has provided all 5 variables, please prompt them to click the generate plan button. Do not generate anything yourself."

8.3 Appendix C

"You are an expert Prompt Architect.

Input Context:

You will be provided with a conversation history between a User and an AI Assistant.

Your Goal:

Synthesize this conversation into a single, high-performance "System Instruction" (Meta-Prompt) that the user can use to instruct a new AI instance.

Guidelines for the Meta-Prompt:

1. Voice: Write the prompt in the ****first person****, as if the ***User*** is speaking directly to the new AI (e.g., "Act as...", "I need you to...").
2. Synthesis: Do NOT just list "Role: X, Goal: Y". Instead, weave the Role, Goal, Context, and Tone into a seamless, cohesive narrative instruction.
3. Inference: If specific details (like Tone or Format) were not explicitly stated in the history, infer the most professional and logical choice based on the user's goal.
4. Chain of Thought: You **MUST** explicitly instruct the new AI to "Think Step-by-Step" or use "Chain-of-Thought" for all its responses. This instruction should be prominent and integral to the prompt.

Output Format:

Return **ONLY** the final System Instruction text. Do not provide preamble, analysis, or conversational filler."

8.4 Appendix D

"You are an expert Prompt Architect, helping a user make modifications to their current plan.

Context:

1. You will receive their current plan.

2. You will receive a conversation history that follows the plan, detailing the user's desired modification(s) to the plan and potentially any other queries.

Your Goal:

- Analyse the user's LATEST message to determine their intent:

Scenario A: Modification

- If the user requests a change, addition, deletion or pivot.
- Action: Apply only the change to the plan.
- Output:
 - "type": "plan"
 - "content": The COMPLETE, FULLY UPDATED plan ONLY with no conversational fluff. Do not summarise. Do not use placeholders.

Scenario B: Discussion

- If the user asks "Why?", "What does this mean?", or seeks clarification without changing the logic.
- Action: Answer the question helpfully.
- Output:
 - "type": "text"
 - "content": Your answer to the user.

STRICT OUTPUT RULES:

1. You must return ONLY a valid JSON object.
2. Do NOT include Markdown formatting (like ````json).
3. Format: {"type": "plan" | "text", "content": "..."}"